

# Complexité Algorithmique

*Basé sur le cours de Pascal KOIRAN  
Notes prises par Hugo SALOU*



*20 avril 2026*

# Table des matières

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| <b>1</b> | <b>Machines de Turing.</b>                               | <b>5</b>  |
| 1.1      | Définitions. . . . .                                     | 5         |
| 1.2      | Non déterminisme. . . . .                                | 8         |
| 1.3      | NP-complétude. . . . .                                   | 9         |
| <b>2</b> | <b>Circuits booléens.</b>                                | <b>11</b> |
| 2.1      | Définitions. . . . .                                     | 11        |
| 2.2      | Simulation de machines de Turing par les circuits. . . . | 12        |
| 2.3      | Premiers problèmes NP-complets. . . . .                  | 15        |
| <b>3</b> | <b>Complexité en espace.</b>                             | <b>17</b> |
| 3.1      | Définitions et premières propriétés. . . . .             | 17        |
| 3.2      | Hierarchie en espace. . . . .                            | 20        |
| 3.3      | Complexité en espace non-déterministe. . . . .           | 22        |
| 3.4      | Formules booléennes quantifiées, PSPACE. . . . .         | 23        |
| 3.5      | Problème PATH et théorème de Savitch. . . . .            | 26        |
| <b>4</b> | <b>Oracles et fonctions de conseil.</b>                  | <b>30</b> |
| 4.1      | Relativisation. . . . .                                  | 31        |
| 4.2      | Fonctions de conseil. . . . .                            | 34        |
| <b>5</b> | <b>Hierarchie polynomiale.</b>                           | <b>36</b> |
| 5.1      | Caractérisation par quantificateurs. . . . .             | 38        |
| 5.2      | Théorème de Karp-Lipton. . . . .                         | 40        |
| <b>6</b> | <b>Algorithmes probabilistes.</b>                        | <b>43</b> |
| <b>7</b> | <b>Protocoles interactifs.</b>                           | <b>52</b> |
| 7.1      | La classe IP. . . . .                                    | 54        |

|          |                                   |           |
|----------|-----------------------------------|-----------|
| 7.2      | Preuve de $IP \subseteq PSPACE$ . | 60        |
| <b>8</b> | <b>Complexité de comptage.</b>    | <b>63</b> |
| 8.1      | Problèmes de comptage.            | 63        |
| 8.2      | La $\#P$ -complétude.             | 65        |
| 8.3      | Comptage modulaire.               | 70        |

Merci Amaury MAZOYER pour les 2.5 premiers chapitres (même si j'ai corrigé quelques coquilles).

# 1 Machines de Turing.

On travaille avec des machines à  $k \geq 1$  rubans. Les rubans sont semi-infinis à droite et sur chaque ruban, on a une tête de lecture qui lit le contenu d'une case.

À chaque étape,

- ▷ la machine  $M$  lit les  $k$  caractères situés sous les têtes de lecture  $(a_1, \dots, a_k)$  ;
- ▷ en fonction de son état interne  $q \in Q$ , et des caractères lus,  $M$  remplace chaque  $a_i$  par  $a'_i$ , déplace les têtes de lectures (d'au plus une case à droite ou à gauche), et passe dans un état  $q' \in Q$ .

## 1.1 Définitions.

■ **Définition 1.1.** Formellement, une *machine de Turing* à  $k$  rubans est un triplet  $(\Gamma, Q, \delta)$  où

- ▷  $\Gamma$  est l'alphabet du ruban ;
- ▷  $Q$  est un ensemble fini d'états ;
- ▷  $\delta : Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{\leftarrow, \rightarrow, \text{I}\}^k$  la fonction de transition.

■ **Remarque 1.1.** On fixe

- ▷ un *ruban d'entrée* (généralement supposé en lecture uniquement),
- ▷ un *ruban de sortie* (généralement supposé en écriture uniquement),
- ▷ un état initial  $q_{\text{initial}}$  ;
- ▷ un état final  $q_{\text{final}}$  ;

- ▷ un symbole blanc  $\square \in \Gamma$  ;
- ▷ un symbole de départ  $\triangleright \in \Gamma$  ;
- ▷ un alphabet d'entrée  $\Sigma \subseteq \Gamma \setminus \{\triangleright, \square\}$ .<sup>1</sup>

Au départ,  $M$  est dans l'état  $q_{\text{initial}}$ , le ruban d'entrée contient le mot infini  $\triangleright x \square^\infty$  où  $x \in \Sigma^*$  est l'entrée.

☞ **Définition 1.2.** On dit que  $M$  *calcule* la fonction  $f : \Sigma^* \rightarrow \Sigma^*$  si, pour toute entrée  $x \in \Sigma^*$ , le calcul de  $M$  termine et  $f(x)$  est écrit sur le ruban de sortie.

On dit qu'un langage  $L \subseteq \Sigma^*$  est *reconnu* si on peut calculer la fonction  $\mathbb{1}_L : \Sigma^* \rightarrow \{\emptyset, 1\} \subseteq \Sigma$ . On peut aussi avoir un état  $q_{\text{accepte}}$  acceptant et un état  $q_{\text{rejet}}$  de rejet.

☞ **Remarque 1.2 (Variantes).** On peut considérer un modèle avec des rubans bi-infinis. C'est un modèle équivalent (c.f. TD).

On peut considérer une machine avec  $\Gamma = \{\emptyset, 1, 2, 3, \square, \triangleright\}$ , c'est-à-dire un alphabet plus gros. Ce modèle est équivalent avec l'association

$$\emptyset \leftrightarrow \emptyset\emptyset \quad 1 \leftrightarrow \emptyset 1 \quad 2 \leftrightarrow 1\emptyset \quad 3 \leftrightarrow 11.$$

☞ **Définition 1.3.** Un langage  $L \subseteq \Sigma^*$  est *reconnu en temps*  $T(n)$  par une machine  $M$  si

- ▷  $M$  reconnaît  $L$  ;
- ▷ sur toute entrée  $x$  de taille  $n$ , la machine  $M$  s'arrête en au plus  $T(n)$  étapes de calcul.

☞ **Définition 1.4.** On dit que  $L$  est dans la classe  $\text{DTIME}(f(n))$  s'il existe une machine (à plusieurs rubans) qui reconnaît  $L$  en temps  $O(f(n))$ . On supposera toujours avoir  $f(n) \geq n + 1$ .<sup>2</sup>

1. Généralement, on prendra  $\Sigma \subseteq \{\emptyset, 1\}$ .

2. Il faut, au moins, lire l'entrée.

▮ **Définition 1.5.** La classe  $\mathbf{P}$  est l'ensemble des langages reconnaissables en temps polynomial :

$$\mathbf{P} = \bigcup_{\alpha \geq 1} \mathbf{DTIME}(n^\alpha).$$

▮ **Théorème 1.1.** Si  $L \in \mathbf{DTIME}(f(n))$  alors  $L$  peut être reconnu en temps  $O(f(n)^2)$  sur une machine à un ruban.

▮ **Théorème 1.2 (Simulation efficace).** Pour toute machine  $M$  fonctionnant en temps  $T(n)$ , il existe une machine  $M'$  à deux rubans qui fonctionne en temps  $O(T(n) \log T(n))$  telle que  $M(x) = M'(x)$  pour toute entrée  $x \in \Sigma^*$ .

▮ **Définition 1.6.** Étant donné  $x, y$ , on définit l'encodage du couple  $(x, y)$  comme le mot  $\langle x, y \rangle = 1^{|x|}0xy$ .

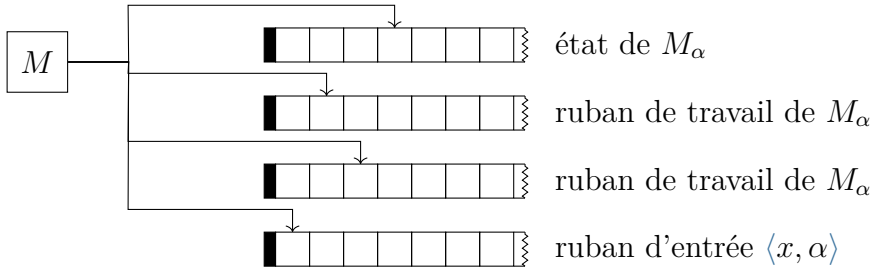
▮ **Définition 1.7.** Une *machine universelle*  $U$  prend en entrée des couples  $\langle x, \alpha \rangle$  (où  $\alpha$  est le code d'une machine  $M_\alpha$  et  $x \in \Sigma^*$ ) et simule la machine  $M_\alpha$  sur l'entrée  $x$ . Autrement dit, pour tout  $x$  et tout  $\alpha$ , on a

$$U(\langle x, \alpha \rangle) = M_\alpha(x).$$

▮ **Théorème 1.3.** Il existe une machine de Turing universelle  $U$  telle que, sur toute entrée  $\langle x, \alpha \rangle$ , si  $M_\alpha$  s'arrête sur  $x$  en  $t$  étapes, alors la machine  $U$  s'arrête sur  $\langle x, \alpha \rangle$  en au plus  $c t \log t$ , où  $c$  ne dépend que de  $x$ .

▮ **Preuve.**

- ▷ *Cas d'une machine à deux rubans.* On montre d'abord que si  $M_\alpha$  est une machine à deux rubans, alors  $U$  peut simuler



**Figure 1.1** | Construction de  $U$  dans le cas à deux rubans

$M_\alpha$  en temps linéaire. En effet, on utilise quatre rubans (figure 1.1) :

- deux rubans pour stocker les deux rubans de  $M_\alpha$  ;
- un ruban qui stocke l'état de  $M_\alpha$  ;
- le ruban d'entrée qui contient  $\langle x, \alpha \rangle$ .

Pour faire une étape de calcul de  $M_\alpha$ , la machine  $U$  doit déterminer  $\delta(q, a, b)$ , mais la complexité de cette opération est cachée dans la constante.

- ▷ *Cas général.* Par le théorème de simulation efficace, on peut construire une machine  $M_\beta$  à deux rubans équivalente et on peut simuler  $M_\beta$  en temps linéaire, et on obtient bien la complexité attendue.

□

## 1.2 Non déterminisme.

■ **Définition 1.8.** Une *machine de Turing non-déterministe* est un triplet  $(\Gamma, Q, \delta)$  où

$$\delta : Q \times \Gamma^k \rightarrow \wp(Q \times \Gamma^k \times \{\leftarrow, \rightarrow, \text{I}\}^k),$$

où l'on distingue deux états  $q_{\text{accepte}}$  et  $q_{\text{rejet}}$ .

Une entrée  $x \in \Sigma^*$  est *acceptée* s'il existe un chemin d'exécution acceptant sur l'entrée  $x$ .

☞ **Définition 1.9.** On note  $\text{NTIME}(f(n))$  l'ensemble des langages acceptés par une machine de Turing non-déterministe fonctionnant en temps  $O(f(n))$  sur toute entrée de taille  $n$  et tout chemin de calcul.

☞ **Définition 1.10.** Un langage  $L$  est dans  $\text{NP}$  s'il existe une machine non-déterministe  $M$  fonctionnant en temps polynomial tel que  $L$  est l'ensemble des entrées acceptées par  $M$ . Ainsi,

$$\text{NP} = \bigcup_{\alpha \geq 1} \text{NTIME}(n^\alpha).$$

☞ **Théorème 1.4** (Définition alternative de  $\text{NP}$ ). Un langage  $L$  est dans  $\text{NP}$  s'il existe un polynôme  $p$  et  $A \in \mathcal{P}$  tel que, pour toute entrée  $x \in \{0, 1\}^*$ ,

$$x \in L \quad \Longleftrightarrow \quad \exists y \in \{0, 1\}^*, \quad \langle x, y \rangle \in A.$$

On dit que  $y$  *certifie* que  $x$  est dans  $L$ .

## 1.3 NP-complétude.

☞ **Définition 1.11.** On dit qu'un problème  $A$  *se réduit à*  $B$  en temps *polynomial* s'il existe une fonction  $f : \Sigma^* \rightarrow \Sigma^*$  calculable en temps polynomial telle que, pour toute entrée  $x$ ,

$$x \in A \quad \Longleftrightarrow \quad f(x) \in B.$$

On note alors  $A \leq_{\mathcal{P}} B$  ou  $A \leq_M B$ .<sup>3</sup>

☞ **Remarque 1.3.** Si  $A \leq_{\mathcal{P}} B$  et  $B \leq_{\mathcal{P}} C$  alors  $A \leq_{\mathcal{P}} C$  par composition des réductions.

---

3. Personnellement, je noterai parfois  $f : A \leq_{\mathcal{P}} B$  où  $f$  est une fonction de réduction.

▮ **Définition 1.12 (NP-complétude).** On dit que  $A$  est **NP-complet** dès lors que

- ▷  $A \in \text{NP}$  ;
- ▷  $A$  est **NP-dur**, c'est-à-dire  $B \leq_P A$  pour tout  $B \in \text{NP}$ .

▮ **Remarque 1.4.** Pour montrer qu'un problème  $A \in \text{NP}$  est **NP-complet**, il suffit de montrer que  $B \leq_P A$  où  $B$  est un problème **NP-complet**. En effet, on a alors, pour tout  $C \in \text{NP}$ ,

$$C \leq_P B \leq_P A.$$

Il suffit donc d'avoir un problème **NP-complet** comme « point de départ ». Généralement, on choisit **SAT** ou **3-SAT**, mais dans ce cours on partira de **CIRCUITSAT** (défini au prochain chapitre).

**SAT** | **Entrée.** Une formule booléenne  $\phi$   
**Sortie.** La formule  $\phi$  est-elle satisfiable ?

**3-SAT** | **Entrée.** Une formule booléenne  $\phi$  sous 3-CNF  
**Sortie.** La formule  $\phi$  est-elle satisfiable ?

# 2 Circuits booléens.

## 2.1 Définitions.

■ **Définition 2.1.** Un *circuit booléen* est un DAG<sup>1</sup> dont les sommets sont étiquetés et de degré entrant 0, 1 ou 2 :

- ▷ les sommets de degré entrant 2 sont étiquetés par  $\wedge$  ou  $\vee$  ;
- ▷ les sommets de degré entrant 1 sont étiquetés par  $\neg$  ;
- ▷ les sommets de degré 0 sont étiquetés par 0, 1 ou des variables booléennes  $x_1, \dots, x_n$ .

Les sommets sont appelés *portes*, et les sommets de degré 0 sont des *portes d'entrées*.

■ **Définition 2.2.** Soit  $C$  un circuit booléen avec des variables d'entrée  $x_1, \dots, x_n$ . Étant donné une *valuation*  $a \in \{0, 1\}^n$ , pour chaque porte  $\alpha$  de  $C$ , on définit la *valeur* prise par  $\alpha$  sur l'entrée  $a$  par :

- ▷ pour les portes d'entrées, on pose

$$\text{val}(x_i) := a_i, \quad \text{val}(0) := 0, \quad \text{val}(1) := 1 ;$$

- ▷ pour les autres portes, on pose
  - $\text{val}(\alpha \vee \beta) = \text{val}(\alpha) \vee \text{val}(\beta)$ ,
  - $\text{val}(\alpha \wedge \beta) = \text{val}(\alpha) \wedge \text{val}(\beta)$ ,
  - $\text{val}(\neg \alpha) = \text{not } \text{val}(\alpha)$ .

---

1. *Directed Acyclic Graph*, un graphe orienté acyclique

▮ **Définition 2.3.** Si  $C$  n'a qu'une seule porte  $\alpha$  de degré sortant 0, on dit que  $\alpha$  est *porte de sortie* de  $C$ , et on pose  $\text{val}(C) := \text{val}(\alpha)$ .

On peut donc dire que  $C$  calcule une fonction  $f : \{0, 1\}^n \rightarrow \{0, 1\}$  lorsque  $f(a)$  est la valeur prise par  $C$  sur l'entrée  $a$ .

Plus généralement, si  $C$  a  $s$  portes de sorties, le circuit  $C$  peut calculer une fonction  $f : \{0, 1\}^n \rightarrow \{0, 1\}^s$ .

▮ **Remarque 2.1.** Toute fonction booléenne peut être calculée par un circuit : il suffit de considérer l'arbre de syntaxe de celle-ci.

▮ **Exercice 2.1.** Donner une famille de fonctions booléennes « explicites »  $(f_n)_{n \geq 1}$  qui ne sont pas calculables par des circuits de taille polynomiale.

▮ **Lemme 2.1.** On a  $\text{VALCIRC} \in \mathbf{P}$  où

|                  |                                                                                                                                  |
|------------------|----------------------------------------------------------------------------------------------------------------------------------|
| $\text{VALCIRC}$ | <b>Entrée.</b> Un circuit booléen $C$ et $a \in \{0, 1\}^n$<br><b>Sortie.</b> Est-ce que $\text{val}(C) = 1$ sous l'entrée $a$ ? |
|------------------|----------------------------------------------------------------------------------------------------------------------------------|

▮ **Preuve.** On utilise l'algorithme suivant :

- 1 : Calculer un tri topologique de  $C$  (pour la boucle ci-dessous).
- 2 : **pour** toute porte  $\alpha$  de  $C$  **faire**
- 3 :     $\lfloor$  Évaluer  $\alpha$  à l'aide des valeurs déjà calculées<sup>2</sup>
- 4 : **retourner** la valeur de la porte de sortie

□

## 2.2 Simulation de machines de Turing par les circuits.

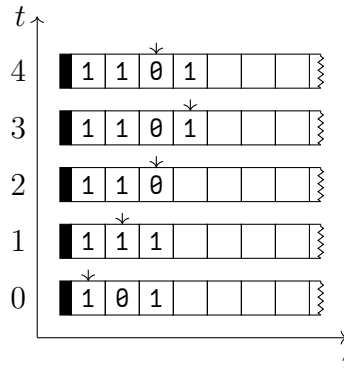
---

2. Par l'ordre topologique, on sait que les entrées de  $\alpha$  ont déjà été évaluées.

**Proposition 2.1.** Soit  $M$  une machine à un ruban fonctionnant en temps  $T(n)$ . On peut simuler  $M$  sur les entrées de taille  $n$  par un circuit de taille  $O(T(n)^2)$ .

**Remarque 2.2.** On peut donner la borne de  $O(T(n) \log T(n))$  au lieu de  $O(T(n)^2)$ , même pour des machines à plusieurs rubans.

**Définition 2.4.** Un *diagramme espace-temps* est un diagramme 2D représentant l'état d'un ruban de  $M$  au cours du temps (figure 2.1)



**Figure 2.1** | Exemple de diagramme espace-temps

**Preuve (Simulation par circuit).** Considérons un diagramme de taille  $p \times p$ , où  $p = T(n) + 1$ . Par la localité du calcul, le contenu de la cellule  $(i, t)$  ne dépend que de trois cellules sur le ruban du dessous : celle directement en dessous, celle en dessous et à gauche, et celle en dessous et à droite.

Définissons une variable  $\ell_{a,i,t}$  qui dit « à l'instant  $t$ , la case  $i$  contient la lettre  $a$  » pour tout  $a, i, t$ . Définissons aussi  $q_{r,i,t}$  indiquant « à l'instant  $t$  la machine est dans l'état  $r$  et la tête de lecture est sur la case  $i$  ».

Comme indiqué, on peut déterminer la valeur de ces variables à partir des valeurs de ces variables avec  $(i-1, t-1)$ ,  $(i, t-1)$  et  $(i+1, t-1)$ . Ceci nous permet d'en déduire une fonction booléenne

$$f : \{0, 1\}^{3p} \rightarrow \{0, 1\}^p.$$

Cette fonction ne dépendant que des transitions de  $M$ , on en déduit un circuit  $C$  calculant  $f$ .

En réalisant  $O(T(n)^2)$  copies de  $C$ , on obtient le circuit final. L'entrée est acceptée si  $\bigvee_{i=0}^{T(n)} q_{\text{accepte}, i, T(n)}$  a pour valeur 1.  $\square$

**Définition 2.5 (Famille de circuits uniforme).** Soit  $(C_n)_{n \geq 1}$  une famille de circuits sur  $n$  variables. Cette famille est *uniforme* s'il existe une machine de Turing qui, sur l'entrée  $1^n$ , calcule une description complète de  $C_n$  en temps polynomial. C'est-à-dire, pour chaque porte  $\alpha$  de  $C_n$ , elle calcule

- ▷ le type de porte ;
- ▷ si c'est une porte d'entrée, son étiquette ;
- ▷ si ce n'est pas une porte d'entrée, les numéros des portes en entrée de  $\alpha$ .

**Remarque 2.3.** Si  $(C_n)_{n \geq 1}$  est une famille uniforme, alors  $C_n$  est de taille polynomiale en  $n$  car sa description complète est écrite en temps polynomial.

**Théorème 2.1.** Un langage  $L \subseteq \{0, 1\}^*$  est dans **P** si et seulement si  $L$  est reconnu par une famille uniforme de circuits booléens de taille polynomiale.

**Preuve.**

- ▷ «  $\implies$  ». Soit  $L \in \mathbf{P}$ . Dans la preuve précédente, on construit une famille de circuits taille polynomiale. Elle est uniforme car il suffit d'itérer sur  $i$  et sur  $t$ , pour calculer les

valeurs de  $\ell_{a,i,t}$  et  $q_{r,i,t}$ . Ceci se fait en temps polynomial.

- ▷ «  $\Leftarrow$  ». Pour tester si  $x \in \{0,1\}^n$  est dans  $L$ , on construit  $C_n$  (en temps polynomial), puis on évalue  $C_n$  sur  $x$  (qui se fait en temps polynomial).

□

## 2.3 Premiers problèmes NP-complets.

▮ **Théorème 2.2.** Le problème CIRCUITSAT est NP-complet où

|            |                                                                                                                 |
|------------|-----------------------------------------------------------------------------------------------------------------|
| CIRCUITSAT | <b>Entrée.</b> Un circuit booléen $C$ avec $n$ variables<br><b>Sortie.</b> Existe-t-il $a$ tel que $C(a) = 1$ ? |
|------------|-----------------------------------------------------------------------------------------------------------------|

▮ **Preuve.**

- ▷ D'une part, CIRCUITSAT  $\in$  NP avec  $a$  le certificat et VAL-CIRC le problème de vérification.
- ▷ D'autre part, soit  $L \in$  NP. Il existe une machine de Turing non-déterministe  $M$  fonctionnant en temps polynômial et qui reconnaît  $L$ . Soit  $x \in \{0,1\}^n$ . On doit construire en temps polynomial un circuit  $C_x$  qui est satisfiable ssi  $x \in L$ .

On peut simuler  $M$  sur l'entrée  $x$  par un circuit de taille  $O(T(n)^2)$ ... mais ce n'est pas suffisant : il faut modéliser le non-déterminisme.

On ajoute des variables d'entrée supplémentaires  $y_1, \dots, y_{T(n)}$  qui modélisent les choix non-déterministes de la machine  $M$ .

On en déduit  $C_x$  (construit en temps polynomial) qui simule  $M$  sur l'entrée  $x$  avec les  $(y_i)_i$  pour les choix non-déterministes.

On a que  $C_x$  est satisfiable si et seulement s'il existe une suite de choix non-déterministes (les valeurs des  $y_i$ ) telle que  $M$  accepte l'entrée  $x$ , c'est-à-dire ssi  $x \in L$ .

□

▮ **Théorème 2.3 (Cook-Levin).** Le problème **3-SAT** est **NP**-complet où

**3-SAT** | **Entrée.** Une formule booléenne  $\phi$  sous 3-CNF  
**Sortie.** La formule  $\phi$  est-elle satisfiable ?

▮ **Preuve.**

- ▷ On a que **3-SAT**  $\in$  **NP** car on peut utiliser la valuation comme certificat.
- ▷ On fait une réduction de **CIRCUITSAT** à **3-SAT**. Soit  $C$  un circuit booléen avec des variables  $x_1, \dots, x_n$ . Pour chaque porte  $\alpha$ , on crée une variable  $z_\alpha$  qui représente la valeur prise par le porte  $\alpha$ . On utilise l'identité  $(P \Rightarrow Q) \Leftrightarrow \bar{P} \vee Q$  pour construire les clauses qui contraignent les variables  $z_\alpha$  à respecter le fonctionnement des portes. Par exemple :

- si  $\alpha = \beta \wedge \gamma$ , on ajoute les clauses

$$\underbrace{(\bar{z}_\beta \vee \bar{z}_\gamma \vee z_\alpha)}_{\beta \wedge \gamma \Rightarrow \alpha}, \quad \underbrace{(z_\beta \vee \bar{z}_\alpha)}_{\alpha \Rightarrow \beta} \quad \text{et} \quad \underbrace{(z_\gamma \vee \bar{z}_\alpha)}_{\alpha \Rightarrow \gamma};$$

- si  $\alpha = \beta \vee \gamma$ , on ajoute les clauses

$$\underbrace{(z_\beta \vee z_\gamma \vee \bar{z}_\alpha)}_{\alpha \Rightarrow \beta \vee \gamma}, \quad \underbrace{(\bar{z}_\beta \vee z_\alpha)}_{\beta \Rightarrow \alpha} \quad \text{et} \quad \underbrace{(\bar{z}_\gamma \vee z_\alpha)}_{\gamma \Rightarrow \alpha};$$

- si  $\alpha = \neg\beta$ , on ajoute les clauses

$$\underbrace{(\bar{z}_\beta \vee \bar{z}_\alpha)}_{\alpha \Rightarrow \bar{\beta}} \quad \text{et} \quad \underbrace{(z_\beta \vee z_\alpha)}_{\bar{\alpha} \Rightarrow \beta}.$$

Enfin, on ajoute la clause  $z_{\alpha_{\text{sortie}}}$  pour forcer la porte de sortie à être vraie.

□

# 3 Complexité en espace.

## 3.1 Définitions et premières propriétés.

▮ **Définition 3.1.** L'espace utilisé par une machine de Turing déterministe sur une entrée  $x$  est le nombre de cases distinctes utilisés sur les rubans de travail<sup>1</sup> au cours de son calcul sur  $x$ .

On dit que  $M$  fonctionne en espace  $s(n)$  si  $M$  s'arrête sur toutes ses entrées et utilise un espace au plus  $s(n)$  sur toute entrée de taille  $n$ .

On note  $\text{DSPACE}(s(n))$  l'ensemble des langages reconnus par une machine de Turing déterministe fonctionnant en espace  $O(s(n))$ .

▮ **Remarque 3.1.** On supposera que, pour le ruban d'entrée, la tête de lecture ne dépassera jamais la fin de l'entrée.

▮ **Exemple 3.1.**

- ▷ Un algorithme naïf pour  $\text{SAT}$  utilise un espace en  $O(n)$ . En effet, il suffit d'énumérer toutes les valuations possibles avec un compteur binaire de taille  $n$ , puis de vérifier si une de ces valuations satisfait la formule.
- ▷ L'addition de deux entiers de taille  $n$  peut s'effectuer en espace  $O(\log n)$ . En effet, il suffit de stocker les positions des bits en cours d'addition et la retenue.

---

1. Pas le ruban d'entrée, ni le ruban de sortie, ceci permet de parler de machine utilisant un espace logarithmique, bien que la taille de l'entrée soit  $n$ .

▮ **Définition 3.2.** On dit que  $t : \mathbb{N} \rightarrow \mathbb{N}$  est *constructible en espace* s'il existe une machine de Turing qui, sur l'entrée  $1^n$  calcule  $1^{t(n)}$  en espace  $O(t(n))$ .

▮ **Proposition 3.1.** On a l'inclusion

$$\text{NTIME}(f(n)) \subseteq \text{DSPACE}(f(n)).$$

▮ **Preuve.** Supposons  $f$  constructible en espace  $(*)$ .

Soit  $L \in \text{NTIME}(f(n))$  et  $M$  une machine non-déterministe reconnaissant  $L$  en temps au plus  $c f(n)$ . On code un chemin de calcul de  $M$  par  $y \in \llbracket 0, R - 1 \rrbracket^{c f(n)}$  où  $R$  désigne le nombre de choix possibles à chaque étape (il dépend de  $M$ ).

- 1 : Calculer  $f(n)$  en espace  $O(f(n))$
- 2 : **pour tout**  $y$  de taille  $c f(n)$  **faire**
- 3 :     Simuler  $M$  sur  $x$  en suivant les choix donnés par  $y$
- 4 :     **si** la simulation accepte **alors**
- 5 :     **Accepter**
- 6 : **Rejeter**

On a un algorithme en espace  $f(n)$  qui teste l'appartenance au langage  $L$ .

Sans l'hypothèse  $(*)$ , on peut obtenir la même inclusion avec une légère modification de l'argument. On fait fonctionner le même algorithme pour des chemins de calcul de longueur  $t = 1, 2, \dots$  jusqu'à ce que la simulation de  $M$  sur l'entrée de  $x$  s'arrête (ce qui arrive forcément si  $x \in L$ ). On s'arrête pour  $t = c f(n)$  au plus.  $\square$

▮ **Proposition 3.2.** On a l'inclusion

$$\text{DSPACE}(s(n)) \subseteq \text{DTIME}(2^{O(s(n))}),$$

dès lors que  $s(n) \geq \log n$ .

▮ **Preuve.** Soit  $N$  tel que, si un calcul prend un temps plus long que  $N$ , alors il boucle. Nous allons montrer que  $N = 2^{O(s(n))}$ . On compte le nombre de configurations distinctes possibles d'une machine  $M$  sur l'entrée  $x$  de taille  $n$ . Une configuration est donnée<sup>2</sup> par :

- ▷ l'état courant (il y a au plus  $|Q|$  possibilités) ;
- ▷ la position de la tête de lecture sur le ruban d'entrée (il y a au plus  $n$  possibilités) ;
- ▷ le contenu des cases utilisées sur les rubans de travail (il y a au plus  $|\Gamma|^{s(n)}$  possibilités) ;
- ▷ la position des têtes de lecture sur les rubans de travail (il y a au plus  $s(n)^k$  possibilités où  $k$  est le nombre de rubans de travail).

D'où la borne annoncée. □

▮ **Remarque 3.2.** A-t-on  $\text{DSPACE}(1) \subseteq \text{DTIME}(1)$  ? Non ! En effet, retourner le dernier caractère de l'entrée se fait en espace  $O(1)$  mais pas en temps  $O(1)$ .

▮ **Définition 3.3.** On définit  $\text{L} = \text{DSPACE}(\log n)$ .

▮ **Corollaire 3.1.** On a les inclusions

$$\text{L} \subseteq \text{P} \subseteq \text{NP} \subseteq \text{PSPACE},$$

où  $\text{PSPACE}$  est l'ensemble des problèmes définis en espace polynomial (défini plus tard). □

---

2. On oublie la position de la tête de lecture sur le ruban de sortie, vu qu'elle n'influe pas le calcul (car écriture uniquement).

▮ **Théorème 3.1.** Si deux fonctions  $f, g : \{0, 1\}^* \rightarrow \{0, 1\}^*$  sont calculables en espace  $s(n) \geq \log n$  alors leur composée  $g \circ f$  est calculable en espace  $O(s(|x|)) + s(|f(x)|)$ .

▮ **Preuve.** L'idée est de calculer un bit de  $f(x)$  uniquement quand on en a besoin pour calculer  $g(f(x))$ .

▮ **Lemme 3.1.** Étant donné  $i$ , on peut calculer le  $i$ -ème bit de  $f(x)$  en espace  $O(s(|x|))$ .

▮ **Preuve.** Faire fonctionner  $M_f$  (la machine calculant  $f$  en espace logarithmique) sans écrire sur le ruban de sortie, mais simplement en comptant le nombre de bits que l'on voulait écrire. Dès lors que  $i$  est atteint, on renvoie le bit courant : c'est le  $i$ -ème bit de  $f(x)$ . □

On utilise ensuite l'algorithme suivant :

```

1 :  $i \leftarrow 0$ 
2 : tant que  $M_g$  ne s'est pas arrêtée faire
3 :   Calculer le  $i$ -ème bit de  $f(x)$  (par le lemme)
4 :   L'écrire sur le ruban d'entrée de  $M_g$ 
5 :   Effectuer un pas de calcul de  $M_g$ 
6 :   si la tête  $\rightarrow$  sur le ruban d'entrée de  $M_g$  alors
7 :      $i \leftarrow i + 1$ 
8 :   sinon si la tête  $\leftarrow$  sur le ruban d'entrée de  $M_g$  alors
9 :      $i \leftarrow i - 1$ 
```

□

▮ **Corollaire 3.2.** Si  $f$  et  $g$  sont calculables en espace  $O(\log n)$  alors  $g \circ f$  est calculable en espace  $O(\log n)$ . □

## 3.2 Hiérarchie en espace.

▮ **Théorème 3.2.** Si  $s$  est constructible en espace et  $s(n) \geq \log n$  alors il existe un langage reconnaissable en espace  $O(s(n))$  mais pas en espace  $o(s(n))$ .

▮ **Preuve.** L'idée est de faire une preuve par diagonalisation. On construit une machine  $D$  qui fonctionne en espace  $O(s(n))$  et qui reconnaît un langage  $A$  différent de tous les langages reconnus en espace  $o(s(n))$ . Pour cela,  $D$  simule une machine tournant en espace au plus  $o(s(n))$  et on a que  $D(\langle M \rangle)$  accepte ssi  $M(\langle M \rangle)$  rejette (et inversement).

▮ **Lemme 3.2.** Si  $L \in \text{DSpace}(s(n))$  alors  $L$  est reconnaissable en  $O(s(n))$  par une machine à un ruban de travail dont l'alphabet est  $\{0, 1, \square, \triangleright\}$ .  $\square$

▮ **Lemme 3.3.** Il existe une machine de Turing universelle  $U$  qui prend en entrée  $\langle M, x \rangle$  avec  $x \in \{0, 1\}^*$  et  $M$  une machine à un ruban de travail et dont l'alphabet est  $\{0, 1, \square, \triangleright\}$ , et qui simule  $M$  sur l'entrée  $x$  en espace  $O(s(n))$  si  $M$  fonctionne en espace  $s(n)$ . On peut même supposer que  $U$  ne possède qu'un seul ruban de travail.  $\square$

À l'aide de ces deux lemmes, on donne l'algorithme suivant (machine  $D$ ).

- 1 : Lire l'entrée  $w \in \{0, 1\}^*$  (notons  $n$  sa taille)
- 2 : Calculer  $s(n)$  en espace  $O(s(n))$
- 3 : Réserver  $s(n)$  cases sur le ruban de travail
- 4 : **si**  $w$  n'est pas de la forme  $\langle M \rangle 10^{*3}$  **alors**
- 5 :    ▮ **Rejeter**
- 6 : Simuler  $M$  sur l'entrée  $w$
- 7 : **si** on dépasse  $2^{2 \cdot s(n)}$  étapes ou  $s(n)$  cases mémoires **alors**
- 8 :    ▮ **Rejeter**
- 9 : **si**  $M$  accepte **alors Rejeter**
- 10 : **sinon Accepter**

On a que la machine  $D$  s'arrête sur toute entrée et fonctionne en espace  $O(s(n))$ .

De plus, si  $B$  est un langage décidable en  $o(s(n))$  par une machine  $M$ , alors  $B$  est différent du langage de  $A$ . En effet,  $D$  simule  $M$  en espace  $o(s(n))$ .

- ▷ Ainsi, il existe  $n_0$  tel que, pour  $n \geq n_0$ ,  $D$  fera une simulation en espace strictement plus petit que  $s(n)$ , donc ne manquera pas d'espace.
- ▷ Aussi, comme  $M$  fonctionne en temps  $2^{o(s(n))}$  alors la simulation sera en temps strictement plus petit que  $2^{s(n)}$  pour  $n \geq n_1$  pour un certain  $n_1$ , donc ne manquera pas de temps.

En posant  $n_2 := \max(n_0, n_1)$ , on a que sur l'entrée  $\langle M \rangle 10^{n_2}$ , la simulation de  $M$  se fera jusqu'au bout et  $D$  donne une réponse différente de  $M$  sur la même entrée. D'où les deux langages  $B$  et  $A$  sont différents.

□

▮ **Corollaire 3.3.** On a les inclusions strictes suivantes :

$$L \subsetneq DSPACE(n) \subsetneq DSPACE(n^2) \subsetneq \dots \subsetneq PSPACE.$$

### 3.3 Complexité en espace non-déterministe.

▮ **Définition 3.4.** Une machine non-déterministe  $M$  fonctionne en espace au plus  $s(n)$  si, sur toute entrée de taille  $n$ , chaque chemin de calcul utilise un espace au plus  $s(n)$  (sur les rubans de travail).

On note  $NSPACE(s(n))$  l'ensemble des langages reconnus par une machine de Turing non-déterministe en espace  $O(s(n))$ .

▮ **Définition 3.5.** On définit  $NL := NSPACE(\log n)$ .

On a prouvé précédemment que  $\text{DSPACE}(s(n)) \subseteq \text{DTIME}(2^{s(n)})$  (lorsque l'on a que  $s(n) \geq \log n$ ). On peut améliorer cette inclusion avec  $\text{NSPACE}(s(n))$ , comme  $\text{DSPACE}(s(n)) \subseteq \text{NSPACE}(s(n))$ .

▮ **Proposition 3.3.** On a

$$\text{NSPACE}(s(n)) \subseteq \text{DTIME}(2^{O(s(n))}),$$

pour  $s(n) \geq \log n$ .

▮ **Preuve.** On considère  $G_x$  le **graphe** (orienté) **des configurations** de  $M$  sur l'entrée  $x$  de taille  $n$ , où  $c_1 c_2 \in E(G_x)$  ssi  $M$  peut passer de la configuration  $c_1$  à la configuration  $c_2$  en un seul pas de calcul.

▮ **Lemme 3.4.** Le graphe  $G_x$  a  $2^{O(s(n))}$  sommets, et on peut le construire en espace  $O(s(n))$ . □

On explore  $G_x$  à partir de la configuration initiale  $c_{\text{initiale}}$  et on accepte si on atteint une configuration d'acceptation. □

▮ **Remarque 3.3.** On n'a pas utilisé l'hypothèse que  $M$  s'arrête sur toutes ses entrées.

## 3.4 Formules booléennes quantifiés, $\text{PSPACE}$ .

▮ **Définition 3.6.** On définit

$$\text{PSPACE} := \bigcup_{k \geq 1} \text{DSPACE}(n^k).$$

▮ **Théorème 3.3 (Savitch).** On a  $\text{NSPACE}(s(n)) \subseteq \text{DSPACE}(s(n)^2)$  si  $s(n) \geq \log n$ .<sup>4</sup>

☞ **Remarque 3.4.** On peut aussi définir la classe **NPSPACE** mais cette classe est égale à **PSPACE** par Savitch. On a donc

$$\mathbf{L} \subseteq \mathbf{NL} \subseteq \mathbf{P} \subseteq \mathbf{NP} \underset{(*)}{\subseteq} \mathbf{PSPACE} \underset{(**)}{\subseteq} \underbrace{\bigcup_{k \geq 1} \mathbf{DTIME}(2^{n^k})}_{\mathbf{EXPTIME}},$$

où

- ▷  $(*)$  est vraie car  $\mathbf{NTIME}(t(n)) \subseteq \mathbf{DSpace}(t(n))$  ;
- ▷  $(**)$  est vraie car  $\mathbf{DSpace}(s(n)) \subseteq \mathbf{DTIME}(2^{O(s(n))})$ .

On sait, de plus, que  $\mathbf{P} \subsetneq \mathbf{EXPTIME}$  par la hiérarchie en temps (variante (vue en TD) de la hiérarchie en espace vue avant). Et, que  $\mathbf{L} \subsetneq \mathbf{PSPACE}$  par le théorème de Savitch et la hiérarchie en espace.

On autorise des quantificateurs universels et existentiels aux formules vues précédemment.

☞ **Exemple 3.2.** Les formules

- ▷  $\forall x \exists y \left( \overbrace{(x \vee y) \wedge (\bar{x} \vee \bar{y})}^{\text{c'est } x \text{ xor } y} \right)$
- ▷  $\exists y \forall x \left( (x \vee y) \wedge (\bar{x} \vee \bar{y}) \right)$

sont des formules booléennes quantifiées. La première est vraie (avec  $y = \bar{x}$ ) mais pas la seconde (avec  $x = y$ ). On suppose que les quantificateurs sont tous en début de formule (forme prénexe).

☞ **Définition 3.7.** On définit le problème **QBF** comme

|     |                                                                                                          |
|-----|----------------------------------------------------------------------------------------------------------|
| QBF | <b>Entrée.</b> Une formule booléenne quantifiée $F$ (close)<br><b>Sortie.</b> Est-ce que $F$ est vraie ? |
|-----|----------------------------------------------------------------------------------------------------------|

---

4. Depuis sa preuve, on n'a jamais réussi à améliorer ce résultat, ni montrer qu'un facteur carré est nécessaire.

**Proposition 3.4.** On a  $\text{QBF} \in \text{PSPACE}$ .

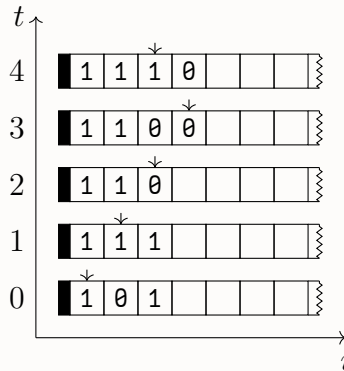
**Preuve.** On utilise l'algorithme suivant.

1. Si  $F$  ne contient pas de quantificateurs, accepter si  $F$  s'évalue à vrai et rejeter si  $F$  s'évalue à faux.
2. Si  $F = \exists x G$ , on décide récursivement avec  $G[x := 0]$  et  $G[x := 1]$ . Si une de ces formules s'évalue à vrai, accepter sinon rejeter.
3. Si  $F = \forall x G$ , on décide récursivement avec  $G[x := 0]$  et  $G[x := 1]$ . Si une de ces formules s'évalue à faux, rejeter sinon accepter.

Cet algorithme utilise un espace linéaire.  $\square$

**Théorème 3.4.** Le problème  $\text{QBF}$  est  $\text{PSPACE}$ -complet.

**Preuve.** Supposons que  $A$  est résolu en espace  $n^k$  par une machine  $M$  à un ruban. On va montrer que  $A \leq_P \text{QBF}$ .



**Figure 3.1** | Exemple de diagramme espace-temps

On considère le diagramme espace-temps de taille  $n^k$  en espace et  $2^{c \cdot n^k}$  en temps. On ne peut pas simplement utiliser la même

preuve que pour **SAT** car le diagramme est exponentiel.

On construit une formule  $\phi_t(c_1, c_2)$  qui exprime qu'on peut aller de la configuration  $c_1$  en la configuration  $c_2$  en au plus  $2^t$  étapes de calcul.

On a donc que  $M$  accepte si  $\phi_{c \cdot n^k}(c_{\text{initiale}}, c_{\text{finale}})$ .<sup>5</sup>

La formule  $\phi_0(c_1, c_2)$  est de taille polynomiale (c.f. preuve du théorème de Cook).

Pour aller de  $\phi_t$  à  $\phi_{t+1}$ , on cherche la configuration du milieu. On pourrait utiliser  $(*) := \exists c \phi_t(c_1, c) \wedge \phi_t(c, c_2)$  qui est correcte mais trop couteuse (taille exponentielle). On choisit plutôt :

$$\exists c \forall c_3 \forall c_4 [(c_3 = c_1 \wedge c_4 = c) \vee (c_3 = c \wedge c_4 = c_2)] \Rightarrow \phi_t(c_3, c_4).$$

Cette formule est logiquement équivalente à  $(*)$  (en regardant les cas où  $(c_3, c_4) = (c_1, c)$  et  $(c_3, c_4) = (c, c_2)$ ). Il est important de se rappeler que  $c, c_3, c_4$  ne sont pas des variables mais des  $n$ -uplets de taille  $O(n^k)$  variables booléennes. La taille de  $\phi_{t+1}$  augmente de  $O(n^k)$  par rapport à la taille de  $\phi_t$ . Au final, on est de taille  $O(n^{2k})$ .<sup>6</sup>  $\square$

Le problème **QBF** est « le » problème **PSPACE**-complet. En TD, on fera des réductions de certains problèmes à **QBF**.

### 3.5 Problème **PATH** et théorème de Savitch.

▮ **Corollaire 3.4.** On a  $\mathbf{NL} \subseteq \mathbf{DSPACE}(\log^2 n)$ .

▮ **Proposition 3.5.** On a que  $\mathbf{PATH} \in \mathbf{DSPACE}(\log^2 n)$ , où

5. On peut considérer qu'il y a une unique configuration acceptante, quitte à transformer tout état acceptant en un état qui efface tout le ruban et remet la tête en position initiale.

6. Le passage quadratique est similaire au théorème de Savitch.

**PATH** | **Entrée.** Un graphe  $G$  orienté, et  $s, t \in V(G)$   
**Sortie.** Existe-t-il un chemin de  $s$  à  $t$  dans  $G$  ?

☞ **Remarque 3.5.** En TD, on a vu que **PATH** est **NL**-complet, et donc la proposition implique le corollaire. En effet, soit  $A \in \mathbf{NL}$  et  $x \in \{0, 1\}^n$ , on a

$$x \in A \iff f(x) \in \mathbf{PATH},$$

où  $f : A \leq_L \mathbf{PATH}$  est la réduction en espace logarithmique (car le problème **PATH** est **NL**-complet). La construction de  $f(x)$  se fait en espace  $O(\log n)$  et décider si  $f(x) \in \mathbf{PATH}$  ou non peut se faire en espace  $O(\log^2 |f(x)|)$ , or  $|f(x)|$  est polynomial en  $|x| = n$ , d'où la borne annoncée.

☞ **Preuve (de la proposition).** On donne un algorithme  $\text{path}(G, u, v, i)$  en espace  $O(\log^2 n)$  qui décide s'il existe, dans  $G$ , un chemin de  $u$  à  $v$  de longueur au plus  $2^i$ . On pourra donc résoudre **PATH** en appelant  $\text{path}(G, s, t, \lceil \log n \rceil)$ .

```

1 : Procédure  $\text{path}(G, u, v, i)$ 
2 :   si  $i = 0$  alors
3 :     si  $uv \in E(G)$  ou  $u = v$  alors Accepter
4 :     sinon Rejeter
5 :   pour tout sommet  $w \in V(G)$  faire
6 :     si  $\text{path}(G, u, w, i - 1)$  et  $\text{path}(G, w, v, i - 1)$  alors
7 :       Accepter
8 :   Rejeter
```

Pour la correction, on montre si  $\text{path}(G, u, v, i)$  accepte alors il existe un chemin de longueur au plus  $2^i$  par récurrence sur  $i$ .

- ▷ Pour le cas  $i = 0$ , c'est bon par le premier « **si** ».
- ▷ Pour l'hérédité, si on a un chemin de longueur au plus  $2^{i-1}$  de  $u$  à  $w$  et un chemin de longueur au plus  $2^{i-1}$  de  $w$  à  $v$ , on concatène ces chemins pour obtenir un chemin de  $u$  à  $v$

de longueur au plus  $2^i$ .

Réciproquement, s'il existe un chemin de  $u$  à  $v$  de longueur au plus  $2^i$ , alors  $\text{path}(G, u, v, i)$  accepte, car il suffit de choisir  $w$  comme sommet milieu du chemin.

On a  $O(\log n)$  appels récurifs ; et à chaque appel, on doit mémoriser  $w$  ce qui demande  $O(\log n)$  bits. On en déduit une complexité en espace en  $O(\log^2 n)$ .  $\square$

☞ **Preuve (du théorème de Savitch).** Supposons que  $A$  peut être résolu par une machine non-déterministe  $M$  en espace  $O(s(n))$  où  $s(n)$  est constructible en espace. Soit  $x \in \{0,1\}^n$  une instance de  $A$ .

On considère le graphe  $G_x$  des **configurations potentielles** de la machine  $M$  sur l'entrée  $x$ , c'est-à-dire l'ensemble des configurations avec  $x$  en entrée et au plus  $c \cdot s(n)$  cases utilisées sur chaque ruban de travail.

☞ **Lemme 3.5.** Le graphe  $G_x$  a  $2^{O(s(n))}$  sommets et peut être construit en espace  $O(s(n))$ .  $\square$

On a que

$$x \in A \quad \Longleftrightarrow \quad (G_x, s, t) \in \text{PATH},$$

où  $s$  est la configuration initiale de  $M$  sur l'entrée  $x$  et  $t$  la configuration acceptante (qu'on supposera unique, *c.f.* preuve de la **PSPACE**-complétude de **QBF**). Par le lemme, on a que  $(G_x, s, t)$  se fait en espace  $O(s(n))$ . Décider si  $(G_x, s, t) \in \text{PATH}$  se fait en espace  $O(\log^2 |G_x|)$  donc  $O(s(n)^2)$ .  $\square$

☞ **Remarque 3.6.** La preuve précédente utilise deux résultats

▷ le théorème de composition *amélioré* :

▮ **Théorème 3.5 (Composition).** Soient  $f$  et  $g$  deux fonctions calculables en espace  $s_f(n)$  (*resp.*  $s_g(n)$ ). On peut calculer la composée  $(f \circ g)(x)$  en espace  $O(s_g(|x|) + s_f(|g(x)|))$ .  $\square$

- ▷ et le fait que l'on pourra supprimer l'hypothèse de constructibilité de  $s(n)$  :

Pour cela, on essaye  $s(n) = 1, 2, \dots$  et on s'arrête à  $s(n) = i$  si aucune configuration de taille  $i + 1$  n'est accessible à partir de la configuration initiale sur l'entrée  $x$  (appel à l'algorithme pour **PATH**).

▮ **Remarque 3.7.** Le problème **PATH** dans les graphes non-orientés est dans **L** ! C'est un résultat récent (2005).

## 4 Oracles et fonctions de conseil.

Une machine de Turing avec un oracle pour  $L$  peut décider en un pas de calcul si un mot donné appartient à  $L$ .

▮ **Définition 4.1.** Une *machine de Turing à oracle* dispose d'un ruban particulier et de trois états particuliers :  $q_?$ ,  $q_{\text{oui}}$  et  $q_{\text{non}}$ .

Soit  $L \subseteq \Sigma^*$ . La machine  $M^L$  se comporte comme une machine de Turing normale, sauf quand elle entre dans l'état  $q_?$ . Dans ce cas, en notant  $u$  le mot sur le ruban de l'oracle,

- ▷ si  $u \in L$  alors l'état suivant est  $q_{\text{oui}}$  ;
- ▷ si  $u \notin L$  alors l'état suivant est  $q_{\text{non}}$ .

▮ **Remarque 4.1.** On peut définir aussi des machines *non déterministes* à oracle.

▮ **Exemple 4.1.**

1. Étant donnée une formule booléenne

$$\phi(x_1, \dots, x_n),$$

on voudrait former une assignation qui satisfait  $\phi$  s'il y en a. On peut le faire avec un oracle pour SAT avec l'algorithme de *recherche préfixe* :

$$a \leftarrow \varepsilon$$

**tant que**  $|a| < n$  **faire**

On demande à l'oracle si  $\phi(a, \emptyset, x_{|a|+2}, \dots, x_n) \in \text{SAT}$ .

**si** l'oracle dit « oui » **alors**  $a \leftarrow a\emptyset$

└ **sinon**  $a \leftarrow a1$

**retourner**  $a$

Si  $\phi \in \text{SAT}$  alors l'algorithme renvoie une assignation qui satisfait  $\phi$ . Sinon, on renvoie  $1^n$ .

1. Soit  $A \subseteq \Sigma^*$ . On voudrait reconnaître

$$L_A := \{1^n \mid A^{\neg n} \neq \emptyset\},$$

où  $A^{\neg n} := \{w \in A \mid |w| = n\} = A \cap \Sigma^n$ .

On peut le faire en temps polynomial avec une machine non-déterministe équipée d'un oracle pour  $A$ .

**si**  $x \neq 1^{|x|}$  **alors Rejeter**

Choisir  $y \in \Sigma^{|x|}$  de manière non-déterministe.

Demander à l'oracle si  $y \in A$ .

**si** l'oracle dit « oui » **alors**

└ **Accepter**

**sinon**

└ **Rejeter**

## 4.1 Relativisation.

On a prouvé les théorèmes de hiérarchie en utilisant la méthode de diagonalisation. On peut étudier les limites de cette méthode grâce à la *relativisation*.

▮ **Théorème 4.1** (Baker, Gill, Solovay). Il existe des oracles  $A$  et  $B$  tels que  $\text{P}^A = \text{NP}^A$  et  $\text{P}^B \neq \text{NP}^B$ .

On prouvera ce théorème dans la suite de cette section.

☞ **Définition 4.2.** Soit  $A$  un oracle.

On note  $P^A$  la classe des langages reconnaissable en temps polynomial par une machine déterministe avec un oracle pour  $A$ .

On note  $NP^A$  la classe des langages reconnaissable en temps polynomial par une machine non-déterministe avec un oracle pour  $A$ .

On étend cette notation avec  $\mathcal{C}$  une classe de langages, on définit

$$P^{\mathcal{C}} := \bigcup_{L \in \mathcal{C}} P^L \quad NP^{\mathcal{C}} := \bigcup_{L \in \mathcal{C}} NP^L.$$

On peut ainsi définir  $P^{NP}$ ,  $NP^{NP}$ , etc.

☞ **Remarque 4.2.** On a  $P \neq EXPTIME$  par le théorème de hiérarchie en temps. La preuve relativise  $P^A \neq EXPTIME^A$  pour tout oracle  $A$ . C'est une propriété des preuves par diagonalisation.

☞ **Remarque 4.3.** L'interprétation du théorème de Baker-Gill-Solovay est que l'on ne peut pas résoudre «  $P$  vs.  $NP$  » par une méthode de diagonalisation.

La preuve que  $IP = PSPACE$  ne se relativise pas : il existe  $A$  tel que  $IP^A \neq PSPACE^A$ .

☞ **Proposition 4.1.** Il existe un oracle  $A$  tel que  $P^A = NP^A$ .

☞ **Preuve.** On choisit  $A = QBF$ , et on a bien  $P^{QBF} = PSPACE$ .

- ▷ On a  $PSPACE \subseteq P^{QBF}$  par  $PSPACE$ -complétude de  $QBF$ .
- ▷ Supposons  $L \in P^{QBF}$ . On a  $L \in PSPACE$  car on peut répondre aux questions posées à l'oracle en espace polynomial puisque  $QBF \in PSPACE$ . D'où  $P^{QBF} \subseteq PSPACE$ .

Et, on a  $NP^{QBF} = PSPACE$ . On doit juste montrer que  $NP^{QBF} \subseteq PSPACE$ . On énumère tous les chemins de calcul d'une ma-

chine  $\text{NP}$ .

□

▮ **Proposition 4.2.** Il existe un oracle  $A$  tel que  $\text{P}^A \neq \text{NP}^A$ .

▮ **Preuve.** Pour tout  $A$ , on note  $L_A := \{1^n \mid A^n \neq \emptyset\}$ . On a que  $L_A \in \text{NP}^A$ . On va construire  $A$  tel que  $L_A \notin \text{P}^A$ .

Soit  $(M_i)_{i \geq 1}$  une énumération des machines à oracle telle que  $M_i$  fonctionne en temps au plus  $p_i(n)$  sur les entrées de taille  $n$ , pour un certain polynôme  $p_i$ .<sup>1</sup> On va construire  $A$  tel que  $L_A$  n'est pas reconnu par  $M_i^A$ . Construisons ce  $A$  par étapes, où l'étape  $i$  assurera que  $L_A$  n'est pas reconnu par  $M_i^A$ .

On part de  $A = \emptyset$ . À chaque étape, on peut ajouter des éléments dans  $A$  ou  $\bar{A}$  : pour un mot  $w \in \Sigma^*$ , à l'étape  $i$ , il est soit dans  $A$ , soit dans  $\bar{A}$ , soit on ne sait pas encore. « À la fin » (étape  $\omega$ ), les mots non-décidés sont mis dans  $\bar{A}$ .

À l'étape  $i$ , on s'assure que  $M_i^A$  donne la mauvaise réponse sur l'entrée  $1^{n_i}$ . On choisit  $n_i$  de sorte que :

1. on ait  $2^{n_i} > p_i(n_i)$  (ce qui est toujours vrai pour un  $n_i$  assez grand) ;
2. l'entier  $n_i$  est strictement supérieur à la longueur de tous les mots pour lesquels on a déjà fait une décision (\*).

On simule  $M_i$  sur l'entrée  $1^{n_i}$ . Pour chaque requête à l'oracle, on donne une réponse consistante avec les décisions précédentes. Plus précisément, on répond « oui » uniquement pour les chaînes  $w$  qu'on a déjà décidé dans  $A$  (sinon, on ne répond « non » et  $w$  est placé dans  $\bar{A}$ ). À la fin du calcul sur  $1^{n_i}$ ,

1. si  $M_i$  rejette, on décide de mettre dans  $A$  un mot de longueur  $n_i$  sur lequel  $M_i$  n'a fait aucune réponse, c'est possible car il y a  $2^{n_i}$  mots de taille  $n_i$  et la machine n'a pu faire que  $p_i(n_i)$  requêtes à l'oracle (et donc il n'y a qu'au plus  $p_i(n_i)$  mots de taille  $n_i$  fixés comme dans  $A$ ) ;

2. si  $M_i$  accepte  $1^{n_i}$ , on décide que  $A^{=n_i} = \emptyset$ , ce qui est consistant avec les décisions précédentes par  $(*)$ , la seconde propriété pour le choix de  $n_i$ .

En conclusion,  $M_i^A(1^{n_i})$  accepte ssi  $A^{=n_i} = \emptyset$ . Et donc  $M_i^A$  se trompe sur  $1^{n_i}$ .  $\square$

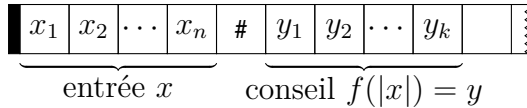
On en conclut le théorème de Baker-Gill-Solovay.

## 4.2 Fonctions de conseil.

**Définition 4.3.** Une *fonction de conseil* est une fonction  $f : \mathbb{N} \rightarrow \{0, 1\}^*$  où, sur l'entrée  $x \in \{0, 1\}^n$  le « conseil »  $f(n)$  est donné à la machine de Turing (en plus de l'entrée).

**Définition 4.4.** Soit  $\mathcal{C}$  une classe de langages, et  $\mathcal{F}$  une classe de fonctions de conseil. On définit  $\mathcal{C}/\mathcal{F}$  l'ensemble des langages  $A$  tels qu'il existe  $B \in \mathcal{C}$  et une fonction de conseil  $f \in \mathcal{F}$  telle que :

$$\forall x \in \{0, 1\}^*, \quad x \in A \iff \langle x, f(|x|) \rangle \in B.$$



**Figure 4.1** | Ruban d'entrée d'une machine avec fonction de conseil<sup>2</sup>

1. On énumère les machines polynomiales à horloge, c'est-à-dire que l'on énumère toutes les machines qui s'arrêtent en temps  $n^k$  pour tout  $k \in \mathbb{N}$ . On force l'arrêt au bout de  $n^k$  étapes de calculs. On parle de « *clocked Turing machines* » en anglais.

**Exemple 4.2.** On note

$$\text{poly} := \left\{ f \mid \begin{array}{l} \text{il existe } p \text{ un polynôme tel} \\ \text{que } |f(n)| \leq p(n) \text{ pour tout } n \end{array} \right\},$$

et

$$\text{log} := \left\{ f \mid \begin{array}{l} \text{il existe } c \text{ une constante telle} \\ \text{que } |f(n)| \leq c \cdot \log n \text{ pour tout } n \geq 1 \end{array} \right\}.$$

On obtient donc des classes  $\text{P/poly}$ ,  $\text{NP/poly}$ ,  $\text{P/log}$ ,  $\text{NP/log}$ , etc.

**Théorème 4.2.** Pour un problème  $A \subseteq \{0, 1\}^*$ , les deux propriétés suivantes sont équivalentes :

1.  $A \in \text{P/poly}$  ;
2.  $A$  peut être résolu par une famille de circuits booléens de taille polynomiale.

**Preuve.** Pour « 1.  $\implies$  2. », soit  $A \in \text{P/poly}$ , et  $x \in \{0, 1\}^n$  avec

$$x \in A \quad \Longleftrightarrow \quad \langle x, f(n) \rangle \in B,$$

où  $B \in \text{P}$ . On a que  $B$  peut être résolu par une famille de circuits booléens  $(C_n)_{n \geq 1}$ . Supposons que  $\langle x, y \rangle = 1^{|x|} 0xy$ . Avec  $y = f(n)$ , cette chaîne est de longueur  $2n+1+|f(n)|$ , et il suffit de considérer  $C_{2n+1+|f(n)|}$ .

Pour « 2.  $\implies$  1. », soit  $A$  résolu par une famille de circuits  $(C_n)_{n \geq 1}$  de taille polynomiale. On donne comme conseil  $f(n)$  une description du circuit  $C_n$ . On peut conclure que  $A \in \text{P/poly}$  puisque le problème d'évaluation  $\text{VALCIRC}$  est dans  $\text{P}$ .  $\square$

---

2. où on construit  $\langle \cdot, \cdot \rangle$  avec un séparateur #.

# 5 Hiérarchie polynomiale.

▮ **Définition 5.1.** Étant donnée une classe de langages  $\mathcal{C}$ , on définit

$$\text{co}\mathcal{C} := \{ A \subseteq \Sigma^* \mid \Sigma^* \setminus A \in \mathcal{C} \}.$$

▮ **Remarque 5.1.** Si  $\mathcal{C} \subseteq \mathcal{C}'$  pour deux classes de langages  $\mathcal{C}$ ,  $\mathcal{C}'$ , alors on a l'inclusion  $\text{co}\mathcal{C} \subseteq \text{co}\mathcal{C}'$ . De plus, on a que  $\text{co}(\text{co}\mathcal{C}) = \mathcal{C}$ .

▮ **Définition 5.2.** Les classes  $\Sigma_i^P$ , pour  $i \geq 0$ , sont définies par induction :

- ▷  $\Sigma_0^P := P$  ;
- ▷  $\Sigma_{i+1}^P := \text{NP}^{\Sigma_i^P}$ .

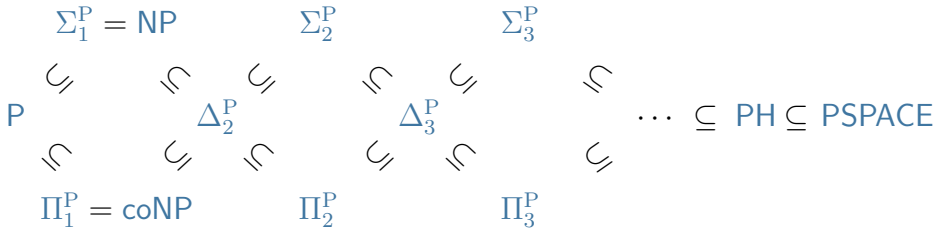
On pose  $\text{PH} := \bigcup_{i \geq 0} \Sigma_i^P$ .

On définit aussi  $\Pi_i^P = \text{co}\Sigma_i^P$  et  $\Delta_i^P := P^{\Sigma_{i-1}^P}$ .

▮ **Exemple 5.1.** On a

- ▷  $\Sigma_1^P = \text{NP}^P = \text{NP}$ ,
- ▷  $\Pi_1^P = \text{coNP}$ ,
- ▷  $\Delta_2^P = P^{\text{NP}}$ ,
- ▷  $\Sigma_2^P = \text{NP}^{\text{NP}}$ ,
- ▷ *etc.*

En général, on a les inclusions suivantes :



▮ **Remarque 5.2.** On a que  $\Delta_i^P = P^{\Sigma_{i-1}^P} = P^{\Pi_{i-1}^P}$ . Par exemple, on a que  $\Delta_2^P = P^{\text{NP}} = P^{\text{coNP}}$ .

Les classes  $\Delta_i^P$  sont closes par complément. Par exemple, l'inclusion  $\Delta_i^P \subseteq \Pi_i^P$  découle de la clôture par complément et de l'inclusion  $\Delta_i^P \subseteq \Sigma_i^P$ .

On omet parfois l'exposant « P », mais attention, il existe une hiérarchie similaire (avec  $\Sigma_i$ ,  $\Pi_i$  et  $\Delta_i$ ) en calculabilité.

Il ne reste que l'inclusion  $\text{PH} \subseteq \text{PSPACE}$  à démontrer.

▮ **Proposition 5.1.** On a  $\text{PH} \subseteq \text{PSPACE}$ .

▮ **Preuve.** On montre par récurrence sur  $i$  que  $\Sigma_i^P \subseteq \text{PSPACE}$ .

- ▷ On a  $\Sigma_0^P = P \subseteq \text{PSPACE}$ .
- ▷ Supposons  $\Sigma_{i-1}^P \subseteq \text{PSPACE}$ . On a

$$\Sigma_i^P = \text{NP}^{\Sigma_{i-1}^P} \subseteq \text{NP}^{\text{PSPACE}} = \text{PSPACE},$$

$$\text{car } \text{NP}^{\text{QBF}} = \text{PSPACE}.$$

□

☞ **Proposition 5.2.** Si  $P = NP$  alors  $PH = P$ .

Plus généralement, pour tout  $i \geq 0$ , si

$$\Sigma_i^P = \Sigma_{i+1}^P,$$

alors  $PH = \Sigma_i^P$ . On dit alors que « la hiérarchie polynomiale s'effondre au  $i$ -ème niveau ».

☞ **Preuve.** Supposons  $\Sigma_i^P = \Sigma_{i+1}^P$ .

On montre par récurrence que  $\Sigma_j^P = \Sigma_i^P$  pour tout  $j \geq i + 1$ . L'initialisation est vraie par hypothèse. L'étape de récurrence est : supposons  $\Sigma_{j-1}^P = \Sigma_i^P$  alors

$$\Sigma_j^P = NP^{\Sigma_{j-1}^P} = NP^{\Sigma_i^P} = \Sigma_{i+1}^P = \Sigma_i^P.$$

□

## 5.1 Caractérisation par quantificateurs.

☞ **Théorème 5.1.** Un langage  $A$  est dans  $\Sigma_i^P$  si, et seulement si, il existe  $B \in P$  et un polynôme  $p$  tel que, pour tout  $x \in \{0, 1\}^*$ ,

$$x \in A \iff \left( \begin{array}{l} \exists y_1 \in \{0, 1\}^{p(n)} \\ \forall y_2 \in \{0, 1\}^{p(n)} \\ \exists y_3 \in \{0, 1\}^{p(n)} \\ \vdots \\ Q_i y_i \in \{0, 1\}^{p(n)} \\ \langle x, y_1, y_2, \dots, y_i \rangle \in B \end{array} \right),$$

avec une alternance de  $\forall$  et de  $\exists$ , commençant par un  $\exists$ .<sup>1</sup>

□

1. On note ici  $Q_i := \exists$  si  $i$  est impair et  $Q_i := \forall$  si  $i$  est pair.

### Remarque 5.3.

1. Cette caractérisation est similaire (c'est une généralisation) à la caractérisation de **NP** avec des certificats.
2. On peut quantifier sur des blocs de taille variables (des chaînes de tailles  $p_1(n), p_2(n), \dots, p_i(n)$ ). On peut aussi enchaîner plusieurs blocs existentiels sans augmenter le  $i$  (il suffit de concaténer les chaînes).
3. On pourrait aussi quantifier sur  $y_k \in \{0, 1\}^{\leq p(n)}$ .
4. On a une caractérisation similaire pour la classe  $\Pi_i^P$  où on commence par «  $\forall y_1 \in \{0, 1\}^{p(n)}$  ».

**Proposition 5.3.** Si  $\Sigma_i^P = \Pi_i^P$  alors on a que  $\Sigma_i^P = \text{PH}$ .

**Preuve.** Montrons  $\Sigma_i^P = \Sigma_{i+1}^P$ . Soit  $A \in \Sigma_{i+1}^P$ . On a

$$x \in A \iff \exists y_1 \forall y_2 \dots \mathbf{Q}_{i+1} y_{i+1} \langle x, y_1, \dots, y_{i+1} \rangle \in B,$$

avec  $B \in \text{P}$ . Et, le langage

$$C := \{ \langle x, y_1 \rangle \mid \forall y_2 \dots \mathbf{Q}_{i+1} y_{i+1} \langle x, y_1, \dots, y_{i+1} \rangle \in B \}$$

est dans  $\Pi_i^P$ , donc dans  $\Sigma_i^P$ . D'où, par caractérisation,

$$x \in A \iff \exists y_1 \underbrace{\exists z_1 \forall z_2 \dots \mathbf{Q}_i z_i \langle x, y_1, z_1, \dots, z_i \rangle \in D}_{\langle x, y_1 \rangle \in C},$$

avec  $D \in \text{P}$ . Et ainsi  $A$  est un problème de  $\Sigma_i^P$  en combinant les deux «  $\exists$  » (avec la remarque précédente).  $\square$

### Remarque 5.4 (Propriétés supplémentaires).

1. La classe  $\Sigma_i^P$  est *close par réduction polynomiale*, c'est-à-dire si  $B \in \Sigma_i^P$  et  $A \leq_P B$  alors  $A \in \Sigma_i^P$ .
2. Le problème de décision **QBF**- $\Sigma_i^P$  est  $\Sigma_i^P$ -complet, où

**QBF- $\Sigma_i^P$**  | **Entrée.** Une formule booléenne quantifiée  $F$  avec  $i$  quantificateurs et commençant par un bloc existentiel  
**Sortie.** Est-ce que  $F$  est vraie ?

3. De même, le problème de décision **QBF- $\Pi_i^P$**  est  $\Pi_i^P$ -complet, où

**QBF- $\Pi_i^P$**  | **Entrée.** Une formule booléenne quantifiée  $F$  avec  $i$  quantificateurs et commençant par un bloc universel  
**Sortie.** Est-ce que  $F$  est vraie ?

## 5.2 Théorème de Karp-Lipton.

▮ **Théorème 5.2** (Karp-Lipton). Si  $\text{NP} \subseteq \text{P/poly}$ , alors  $\Sigma_2^P = \Pi_2^P$ .

▮ **Définition 5.3.** Un circuit booléen à  $s$  entrées *décide SAT* si, étant donnée une formule booléenne  $F$  de taille  $s$ , le circuit  $C$  décide si  $F$  est satisfiable.

La preuve de ce théorème repose sur deux lemmes.

▮ **Lemme 5.1.** L'ensemble des (codages de) circuits qui décident SAT est dans  $\text{coNP}$ .

▮ **Preuve.** On utilise le fait que SAT est *auto-réductible*<sup>2</sup> : une formule booléenne  $F(v_1, \dots, v_n)$  est satisfiable si et seulement si l'une des deux formules booléennes

$$F(v_1, \dots, v_{n-1}, \theta) \quad \text{ou} \quad F(v_1, \dots, v_{n-1}, 1)$$

est satisfiable.

Un circuit  $C$  décide SAT ssi pour toute formule  $F$  de taille  $s$

1. si  $F$  n'a pas de variable, alors  $C(F) = 1$  ssi  $F \equiv 1$  ;

2. si  $F$  dépend de  $n \geq 1$  variables  $v_1, \dots, v_n$  alors  $C(F) = 1$  ssi

$$C(F[v_n := 0]) = 1 \quad \text{ou} \quad C(F[v_n := 1]) = 1.$$

Étant donnée  $F$ , les conditions ci-dessous peuvent être vérifiées en temps polynomial (car **VALCIRC** est dans **P**).

Cette caractérisation commence par un « pour toute formule » et on considère ensuite un problème dans **P**, d'où le langage est bien dans **coNP**.  $\square$

☞ **Lemme 5.2.** Si  $\mathbf{NP} \subseteq \mathbf{P/poly}$ , alors **SAT** peut être décidé par une famille de circuits booléens de taille polynomiale.  $\square$

☞ **Preuve (du théorème de Karp-Lipton).** On suppose avoir l'inclusion des classes  $\mathbf{NP} \subseteq \mathbf{P/poly}$ . Il suffit de montrer que  $\Pi_2^P \subseteq \Sigma_2^P$ . En effet, avec ça on a que

$$\Sigma_2^P = \mathbf{co}\Pi_2^P \subseteq \mathbf{co}\Sigma_2^P = \Pi_2^P.$$

Il suffit de montrer que le problème de décision **QBF- $\Pi_2^P$**  est dans  $\Sigma_2^P$ . Soit  $F$  une formule booléenne de taille  $s$ , alors

$$\forall u \exists v \quad F(u, v),$$

est équivalente à

$$\exists C \forall u \quad C(F(u, \cdot)) = 1 \quad \text{et} \quad C \text{ décide } \mathbf{SAT},$$

où  $C$  est un circuit booléen avec  $s$  entrées. Il suffit de quantifier sur des circuits de taille polynomiale d'après le lemme 5.2. Ceci

---

2. *self-reducible* en anglais.

est équivalent à

$$\exists C \forall u \quad C(F(u, \cdot)) = 1 \quad \text{et} \quad \forall y \in \{\emptyset, 1\}^{p(s)} \langle C, y \rangle \in A,$$

avec  $A \in \mathbf{P}$  d'après le lemme 5.1 et la caractérisation de  $\mathbf{coNP}$ .  
On en déduit que ceci est équivalent à

$$\exists C \forall u \forall y \quad \underbrace{C(F(u, \cdot)) = 1 \quad \text{et} \quad \langle C, y \rangle \in A}_{\text{vérifiable en temps polynomial}},$$

et est donc vérifiable dans  $\Sigma_2^{\mathbf{P}}$  grâce à la caractérisation par quantificateurs.  $\square$

# 6 Algorithmes probabilistes.

Dans ce chapitre, on s'intéresse aux algorithmes probabilistes et les classes de complexité correspondantes. Le modèle de calcul sera les *machines de Turing probabilistes*, qui ressemblent aux machines de Turing non-déterministes, mais pas exactement. Plus précisément, on aura deux fonction de transitions  $\delta_0$  et  $\delta_1$  et, à chaque étape, on applique  $\delta_0$  (ou  $\delta_1$ ) avec probabilité  $1/2$ . De plus, tous ces choix sont indépendants.

▮ **Remarque 6.1.** Les machines de Turing déterministes sont des cas particuliers car il suffit de poser  $\delta_0 := \delta =: \delta_1$ , en notant  $\delta$  la fonction de transition de la machine de Turing déterministe.

▮ **Définition 6.1.** La *probabilité d'accepter*  $w \in \Sigma^*$  est

$$\sum_{\substack{b \text{ chemin de calcul} \\ \text{acceptant sur l'entrée } w}} \Pr[b],$$

où, en notant  $k$  la longueur du chemin,<sup>1</sup>

$$\Pr[b] := \frac{1}{2^k}.$$

▮ **Définition 6.2.** Pour  $0 \leq \varepsilon < 1/2$ , une machine probabiliste *reconnait* un langage  $A$  avec probabilité d'erreur  $\varepsilon$  si

---

1. Dans ce cas, c'est le nombre de tirages au sort.

1. si  $w \in A$  alors  $\Pr[w \text{ est accepté}] \geq 1 - \varepsilon$  ;
2. si  $w \notin A$  alors  $\Pr[w \text{ est rejeté}] \geq 1 - \varepsilon$ .

On considère ensuite les machines probabilistes qui fonctionnent en temps polynomial en  $|w|$ . La machine s'arrêtera donc en un temps polynomial *peu importe* les choix probabilistes.

▮ **Définition 6.3.** On définit la classe **BPP**<sup>2</sup> comme la classe des langages qui peuvent être reconnus par des machines probabilistes qui fonctionnent en temps polynomial avec probabilité d'erreur  $1/3$ .

De manière analogue à la classe **NP**, on a une caractérisation par certificats pour **BPP**.

▮ **Proposition 6.1.** Un langage  $A$  est dans **BPP** si, et seulement si, il existe un polynôme  $p$  et un problème auxiliaire  $B \in \mathbf{P}$  tel que

1. si  $x \in A$  alors  $\Pr_{y \in \{0,1\}^{p(|x|)}} [\langle x, y \rangle \in B] \geq \frac{2}{3}$  ;
2. si  $x \notin A$  alors  $\Pr_{y \in \{0,1\}^{p(|x|)}} [\langle x, y \rangle \notin B] \geq \frac{2}{3}$ .

Le choix de  $y \in \{0,1\}^{p(|x|)}$  se fait avec la distribution uniforme. □

Le choix de  $\varepsilon = 1/3$  dans la définition de **BPP** peut sembler arbitraire mais, mais ça n'a pas d'importance comme le montre le lemme ci-dessous.

▮ **Lemme 6.1 (d'amplification).** Soit  $0 \leq \varepsilon < 1/2$ . Supposons qu'un langage  $A \subseteq \Sigma^*$  est reconnu avec probabilité d'erreur  $\varepsilon$  par une machine de Turing probabiliste polynomiale  $M_1$ . Pour tout polynôme  $p(n)$ , alors  $A$  peut être reconnu par une machine probabiliste polynomiale  $M_2$  avec probabilité d'erreur  $2^{-p(n)}$  sur toute entrée de taille  $n$ .

**Preuve.** L'algorithme de  $M_2$  est de la forme suivante.

```

1 : Effectuer  $2k$  calculs de  $M_1$  sur l'entrée  $x \in \Sigma^n$ 
2 : si  $M_1$  accepte pour une majorité (stricte) de calculs alors
3 :   | Accepter
4 : sinon
5 :   | Rejeter
```

Il nous reste à déterminer la valeur de  $k$  telle que

$$\Pr[M_2 \text{ fasse une erreur sur l'entrée } x] \leq \frac{1}{2^{p(n)}}.$$

Pour  $x$  donné, soit  $S \in \{0, 1\}^{2k}$  la suite des résultats obtenus à la ligne 1. Notons  $c$  le nombre de résultats corrects et  $w$  le nombre de résultats incorrects (on a donc  $c + w = 2k$ ). On dit que  $S$  est « mauvaise » si  $c \leq w$  (autrement dit  $w \geq k$ ).

On a

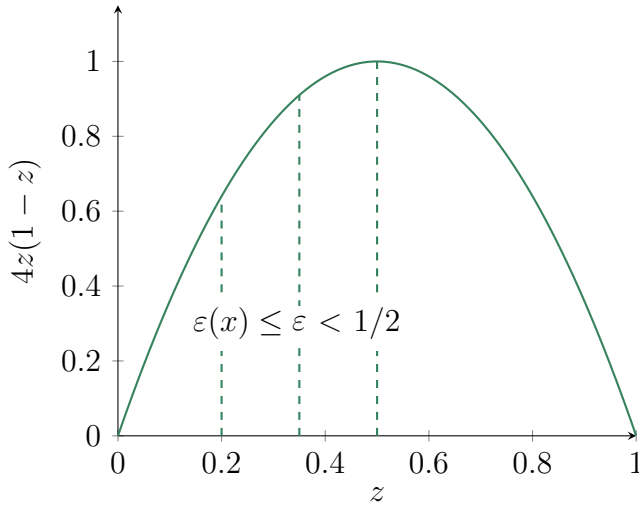
$$\Pr[M_2 \text{ fasse une erreur sur l'entrée } x] \leq \sum_{S \text{ mauvaise}} \Pr[S].$$

On majore (grossièrement) le nombre de mauvaises suites  $S$  par le nombre total de suites, i.e.  $2^{2k}$ . En fixant une mauvaise suite  $S \in \{0, 1\}^{2k}$ , on a

$$\Pr[S] = \varepsilon(x)^w \cdot (1 - \varepsilon)^c = \varepsilon(x)^k \cdot (1 - \varepsilon)^k \cdot \left( \frac{\varepsilon(x)}{1 - \varepsilon(x)} \right)^{w-k},$$

en notant  $\varepsilon(x)$  la probabilité que  $M_1$  fasse une erreur sur l'entrée  $x$  (on a donc  $\varepsilon(x) \leq \varepsilon$ ). Comme  $w \geq k$  (car  $S$  supposée mauvaise), on a

$$\left( \frac{\varepsilon(x)}{1 - \varepsilon(x)} \right)^{w-k} < 1.$$



**Figure 6.1** | Graphe de  $z \mapsto 4z(1-z)$

On en déduit  $\Pr[S] \leq (\varepsilon(x)(1 - \varepsilon(x)))^k$ , d'où

$$\begin{aligned} \Pr[M_2 \text{ fasse une erreur sur l'entrée } x] &\leq (4\varepsilon(x)(1 - \varepsilon(x)))^k \\ &\leq (4\varepsilon(1 - \varepsilon))^k. \end{aligned}$$

Il suffit ainsi d'avoir  $[4\varepsilon(1 - \varepsilon)]^k \leq 2^{-p(n)}$ , autrement dit,

$$k \geq \frac{p(n)}{-\log_2[4\varepsilon(1 - \varepsilon)]}.$$

□

▮ **Proposition 6.2.** On a  $\text{BPP} \subseteq \text{PSPACE}$ .

▮ **Preuve.** On énumère tous les chemins de calcul possible, et on accepte si une majorité de ces chemins accepte. □

▮ **Théorème 6.1.** On a  $\text{BPP} \subseteq \text{P/poly}$ .

▮ **Preuve.** Soit  $A \in \text{BPP}$  et  $B \in \text{P}$  tel que

1. si  $x \in A$  alors  $\Pr_{y \in \{0,1\}^{p(|x|)}} [\langle x, y \rangle \in B] \geq 1 - \varepsilon(|x|)$
2. si  $x \notin A$  alors  $\Pr_{y \in \{0,1\}^{p(|x|)}} [\langle x, y \rangle \notin B] \geq 1 - \varepsilon(|x|)$

où on va choisir  $\varepsilon(|x|)$  exponentiellement petit (par lemme d'amplification).

Pour chaque  $n$ , on va fixer une valeur particulière  $y_n \in \{0,1\}^{p(n)}$  pour la chaîne aléatoire  $y$ . Ce choix définit le conseil pour les entrées de taille  $n$ .

Montrons qu'il existe  $y_n$  qui donne une bonne réponse pour tout  $x \in \{0,1\}^n$ . On pose

$$f(x, y) := \begin{cases} 1 & \text{si } y \text{ conduit au bon résultat sur l'entrée } x \\ 0 & \text{sinon.} \end{cases}$$

On peut représenter ceci en un tableau à double entrées avec  $2^n$  lignes et  $2^{p(n)}$  colonnes. Notre but est donc de trouver une colonne qui ne contient que des « 1 ». On va supposer que la probabilité d'erreur est  $\varepsilon(n) < 1/2^n$ . Dans chaque ligne, on a moins (strict) de  $2^{p(n)}/2^n$  zéros. Dans l'ensemble du tableau, on a moins (strict) de

$$\underbrace{2^n}_{\text{\#lignes}} \times \frac{2^{p(n)}}{2^n} = 2^{p(n)}$$

zéros. On a  $2^{p(n)}$  colonnes donc il existe une colonne qui ne contient pas de zéros.  $\square$

▮ **Théorème 6.2.** On a  $\text{BPP} \subseteq \Sigma_2^{\text{P}} \cap \Pi_2^{\text{P}}$ .

▮ **Preuve.** Il suffit de montrer que  $\text{BPP} \subseteq \Sigma_2^{\text{P}}$  car  $\text{coBPP} = \text{BPP}$ . Soit  $A \in \text{BPP}$ . Il existe donc  $p$  un polynôme et  $B \in \text{P}$  tel que, pour tout  $x \in \Sigma^n$  (on prend une probabilité d'erreur de  $1/2^n$  par

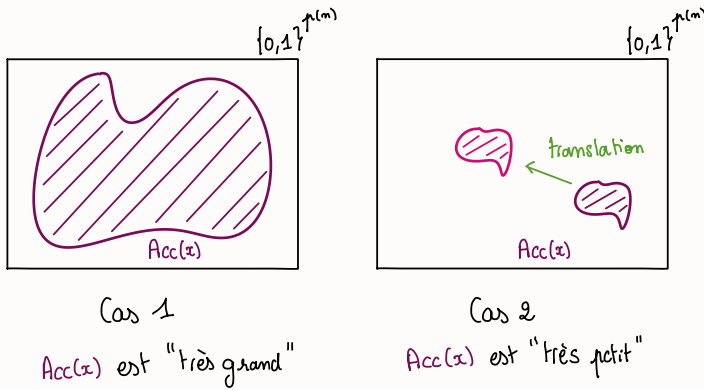
le lemme d'amplification),

1. si  $x \in A$  alors  $\#Acc(x) \geq \left(1 - \frac{1}{2^n}\right) \cdot 2^{p(n)}$  ;
2. si  $x \notin A$  alors  $\#Acc(x) \leq 2^{p(n)}/2^n$ ,

en notant  $Acc(x) := \{y \in \{0,1\}^{p(|x|)} \mid \langle x, y \rangle \in B\}$ .

Fixons  $x \in \{0,1\}^n$ . On doit donc distinguer (avec une formule du problème  $QBF\text{-}\Sigma_2^P$ ) entre deux possibilités :

1. l'ensemble  $Acc(x)$  est « très grand » par rapport à  $\{0,1\}^{p(n)}$  ;
2. l'ensemble  $Acc(x)$  est « très petit » par rapport à  $\{0,1\}^{p(n)}$ .



**Figure 6.2** | Les deux cas à distinguer

L'idée est qu'on essaie de couvrir  $\{0,1\}^{p(n)}$  par un petit nombre de translations de  $Acc(x)$ , où la translation par  $t \in \{0,1\}^{p(n)}$

$$\begin{aligned} \{0,1\}^{p(n)} &\longrightarrow \{0,1\}^{p(n)} \\ x &\longmapsto x \oplus t, \end{aligned}$$

où  $\oplus$  est le XOR bit à bit. On pourra couvrir  $\{0,1\}^{p(n)}$  uniquement dans le cas 1.

On va écrire

$$(*) \quad x \in A \iff \exists t_1 \dots t_m \in \{\emptyset, 1\}^{p(n)} \\ \forall z \in \{\emptyset, 1\}^{p(n)} \quad \bigvee_{i=1}^m \langle x, z \oplus t_i \rangle \in B,$$

où les  $t_1, \dots, t_m$  représentent les  $m$  vecteurs de translation.

Si cette équivalence est vraie avec  $m$  polynomial en  $n$ , on aura montré  $A \in \Sigma_2^P$ . En effet, pour un certain  $z$ , on a

$$\langle x, z \oplus t_i \rangle \in B \iff z \oplus t_i \in \text{Acc}(x) \iff z \in \text{Acc}(x) \oplus t_i,$$

où  $\text{Acc}(x) \oplus t_i$  est le translaté de  $\text{Acc}(x)$ . La formule  $(*)$  dit donc que les  $m$  translations de  $\text{Acc}(x)$  recouvrent  $\{\emptyset, 1\}^{p(n)}$ .

▮ **Lemme 6.2.** Soit  $E \subseteq \{\emptyset, 1\}^k$ . S'il existe  $t_1, \dots, t_m \in \{\emptyset, 1\}^k$  tels que

$$\bigcup_{i=1}^m (E \oplus t_i) = \{\emptyset, 1\}^k,$$

alors  $\#E \geq 2^k/m$ .

▮ **Preuve.** On a

$$\#\left(\bigcup_{i=1}^m (E \oplus t_i)\right) \leq \sum_{i=1}^m \#(E \oplus t_i) = m \#E.$$

□

▮ **Lemme 6.3.** Si  $2^k(1 - \#E/2^k)^m < 1$  alors il existe  $t_1, \dots, t_m$  tels que  $\bigcup_{i=1}^m (E \oplus t_i) = \{\emptyset, 1\}^k$ .

☞ **Remarque 6.2.** On a

$$2^k \left(1 - \frac{\#E}{2^k}\right)^m < 1 \quad \Longleftrightarrow \quad k \ln 2 + m \ln \left(1 - \frac{\#E}{2^k}\right) < 0,$$

donc il suffit d'avoir  $k \ln 2 - m\#E/2^k < 0$ , autrement dit

$$m > \frac{k \ln 2}{\#E/2^k}.$$

☞ **Preuve.** On prend  $t_1, \dots, t_m$  tirés au hasard suivant la distribution uniforme sur  $\{0, 1\}^k$  (choix indépendants et identiquement distribués, « iid »). Soit

$$\rho := \Pr \left[ \bigcup_{i=1}^m (E \oplus t_i) \neq \{0, 1\}^k \right]$$

la probabilité d'erreur. Si  $\rho < 1$  alors on a le résultat. On remarque que  $\rho \leq 2^k(1 - \#E/2^k)^m$ . Puis, on a

$$\begin{aligned} \rho &= \Pr \left[ \bigcup_{z \in \{0, 1\}^k} \left( z \notin \bigcup_{i=1}^m (E \oplus t_i) \right) \right] \\ &\leq \sum_{z \in \{0, 1\}^k} \Pr \left[ z \notin \bigcup_{i=1}^m (E \oplus t_i) \right] \\ &= \sum_{z \in \{0, 1\}^k} \Pr \left[ \bigcap_{i=1}^m (z \notin E \oplus t_i) \right] \\ &= \sum_{z \in \{0, 1\}^k} \prod_{i=1}^m \underbrace{\Pr[t_i \notin E \oplus z]}_{1 - \#E/2^k}, \end{aligned}$$

et on conclut directement. □

On peut donc appliquer les deux lemmes avec  $E = \text{Acc}(x)$ ,  $k = p(n)$  et  $m = 2k$ . On a que

- ▷ si  $x \in A$  alors, par hypothèse  $\#E/2^k \geq 1 - 1/2^n \geq 1/2$  (pour  $n \geq 1$ ), par le second lemme, on a

$$m = 2k > 2k \ln 2 \geq \frac{k \ln 2}{\#E/2^k} ;$$

- ▷ si  $x \notin A$  alors, par hypothèse  $\#E/2^k \leq 1/2^n$ , or pour tout couvrir, on devrait avoir

$$2p(n) \geq \frac{2^{p(n)}}{\#E} \geq 2^n ,$$

ce qui est absurde.

□

# 7 Protocoles interactifs.

Deux machines discutent : le *prouveur*  $\mathcal{P}$  et le *vérificateur*  $\mathcal{V}$ . Le prouveur a une puissance de calcul illimitée, et le vérificateur ne fait pas initialement « confiance » au prouveur.

☞ **Exemple 7.1.** On donne un protocole interactif pour SAT.

---

**Algorithme 1** Protocole interactif pour SAT

---

| Vérificateur $\mathcal{V}$            | Prouveur $\mathcal{P}$                                               |
|---------------------------------------|----------------------------------------------------------------------|
|                                       | 1 : Calculer $(a_1, \dots, a_n)$ une assignation qui satisfait $F$ . |
|                                       | $\xleftarrow{a_1, \dots, a_n}$                                       |
| 1 : si $F(a_1, \dots, a_n) = 1$ alors |                                                                      |
| 2 :   <b>Accepter</b>                 |                                                                      |
| 3 : <b>sinon</b>                      |                                                                      |
| 4 :   <b>Rejeter</b>                  |                                                                      |

---

Analysons ce protocole :

- ▷ si  $F \notin \text{SAT}$  alors  $\mathcal{V}$  rejette toujours, peu importe  $\tilde{\mathcal{P}}$  ;
- ▷ si  $F \in \text{SAT}$  alors  $\mathcal{P}$  peut convaincre  $\mathcal{V}$  d'accepter.

☞ **Exemple 7.2.** On donne un protocole interactif pour le *non-isomorphisme de graphes*.

On définit le problème **ISOGRAPH** :

ISOGRAPH

**Entrée.** Deux graphes  $G_0$  et  $G_1$ **Sortie.** Est-ce que  $G_0$  est isomorphe à  $G_1$  ?

où un *isomorphisme* entre  $(W, E)$  et  $(W, E') = \pi(W, E)$  est une permutation  $\pi : W \rightarrow W$  des sommets avec

$$E' := \{ (i, j) \mid (\pi(i), \pi(j)) \in E \}.$$

On notera alors  $(W, E) \cong (W, E')$ .

On a que **ISOGRAPH**  $\in$  **NP** (avec  $\pi$  le certificat). Mais, on ne sait pas si le problème est **NP**-complet, il existe un algorithme en temps  $n^{O(\log n)}$ . (Par définition, le problème de *non*-isomorphisme de graphe est dans **coNP**.)

On donne un protocole interactif pour le problème de non-isomorphisme de graphe. En entrée,  $G_0, G_1$  est une paire de graphes sur un même ensemble de sommets  $W$ .

Le but de  $\mathcal{P}$  est de convaincre  $\mathcal{V}$  que  $G_0 \not\cong G_1$ .

- ▷ Si  $G_0 \not\cong G_1$  alors  $G'$  est isomorphe à un unique graphe  $G_{b'}$  et renvoie  $b'$  au vérificateur.
- ▷ Si  $G_0 \cong G_1$ , alors pour tout prouveur,  $\Pr[\mathcal{V} \text{ accepte}] = 1/2$ . En effet, pour tout  $G'$ ,

$$\Pr[\tilde{\mathcal{P}} \text{ renvoie } G' \mid b = 0] = \Pr[\tilde{\mathcal{P}} \text{ renvoie } G' \mid b = 1],$$

notons  $p_{i,j} := \Pr[\tilde{\mathcal{P}} \text{ envoie } b' = j \mid b = i]$  et on observe que  $p_{11} = p_{21}$ , d'où

$$\Pr[\mathcal{V} \text{ accepte}] = \frac{1}{2}(p_{11} + p_{22}) = \frac{1}{2}(p_{21} + p_{22}) = \frac{1}{2}.$$

**Algorithme 2** Protocole interactif pour  $\Sigma^* \setminus \text{ISOGRAPH}$ 

| Vérificateur $\mathcal{V}$                            | Prouveur $\mathcal{P}$                                                                                                                                                                                                  |
|-------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Entrée</b> $G_0, G_1$                              | <b>Entrée</b> $G_0, G_1$                                                                                                                                                                                                |
| 1 : Choisir un bit $b \in \{0, 1\}$<br>aléatoire      |                                                                                                                                                                                                                         |
| 2 : Choisir une permutation $\pi$<br>de $W$ aléatoire |                                                                                                                                                                                                                         |
| 3 : Calculer $G' := \pi(G_b)$                         |                                                                                                                                                                                                                         |
|                                                       | $\xrightarrow{G'}$<br>1 : <b>si</b> $G_0 \not\cong G_1$ <b>alors</b><br>2 :     Le graphe $G'$ est iso-<br>morphe à un unique $G_{b'}$ .<br>3 : <b>sinon</b><br>4 :     Soit $b' \in \{0, 1\}$ choisi<br>aléatoirement. |
|                                                       | $\xleftarrow{b'}$<br>4 : <b>si</b> $b = b'$ <b>alors</b><br>5 : <b>Accepter</b><br>6 : <b>sinon</b><br>7 : <b>Rejeter</b>                                                                                               |

On peut réduire l'erreur en répétant le protocole  $k$  fois, et on accepte si  $(G_0, G_1)$  est toujours accepté par  $\mathcal{V}$ , et on aura  $\Pr[\mathcal{V} \text{ accepte}] \leq 1/2^k$ .

## 7.1 La classe IP.

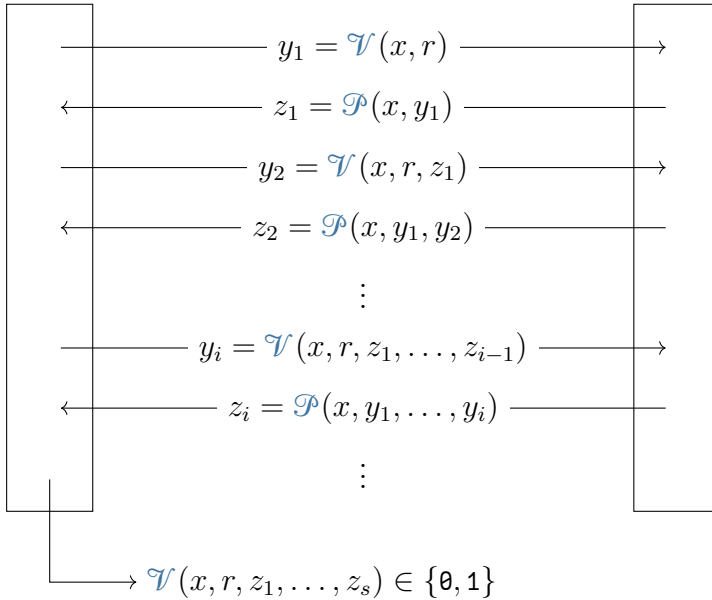
On a la situation représentée en figure 7.1, où

- ▷ le  $x$  est l'entrée du problème (de taille  $n$ ) ;
- ▷ le  $r$  est la donnée des choix aléatoires de  $\mathcal{V}$  (de taille polynomiale en  $n$ ) ;
- ▷ les  $y_i$  sont les messages de  $\mathcal{V}$  à  $\mathcal{P}$  (de taille polynomiale en  $n$ ) ;
- ▷ les  $z_i$  sont les messages de  $\mathcal{P}$  à  $\mathcal{V}$  (de taille polynomiale en  $n$ ) ;
- ▷ le  $s$  est la longueur de l'échange (avec  $s$  polynomial en  $n$ ).

La puissance de calcul de  $\mathcal{P}$  est *non-bornée*, et celle de  $\mathcal{V}$  est polynomialement bornée. La *décision finale du vérificateur* est  $\mathcal{V}(x, r, z_1, \dots, z_s) \in \{0, 1\}$ .

**Vérificateur**  
 $\mathcal{V}$

**Prouveur**  
 $\mathcal{P}$



**Figure 7.1** | *Protocole interactif*

**Définition 7.1.** On définit  $\mathbf{IP}$  la classe des langages que l'on peut reconnaître à l'aide d'un protocole interactif avec probabilité d'erreur inférieur à  $1/3$ . Autrement dit, un langage  $L$  est dans  $\mathbf{IP}$  ss'il existe un vérificateur  $\mathcal{V}$  et un prouveur  $\mathcal{P}$  tels que

1. si  $x \in L$  alors  $\Pr[\mathcal{P} \text{ pousse } \mathcal{V} \text{ à accepter}] \geq \frac{2}{3}$ ;

2. si  $x \notin L$  alors pour tout prouveur  $\tilde{\mathcal{P}}$ ,

$$\Pr[\tilde{\mathcal{P}} \text{ pousse } \mathcal{V} \text{ à accepter}] \leq \frac{1}{3}.$$

▮ **Exemple 7.3.** On a  $\text{SAT} \in \text{IP}$  et  $\Sigma^* \setminus \text{ISOGRAPH} \in \text{IP}$ .

▮ **Théorème 7.1.** On a  $\text{IP} = \text{PSPACE}$ .

▮ **Remarque 7.1.** On a  $\text{IP} \subseteq \text{PSPACE}$ , c'est le « sens facile ».

En revanche, le résultat  $\text{PSPACE} \subseteq \text{IP}$  *ne se relativise pas* : il existe  $A$  tel que  $\text{PSPACE}^A$  n'est pas inclus dans  $\text{IP}^A$  (et même  $\text{coNP}^A$  n'est pas inclus dans  $\text{IP}^A$ ).

Pour montrer  $\text{PSPACE} \subseteq \text{IP}$ , on utilise le fait que  $\text{QBF}$  est  $\text{PSPACE}$ -complet : il suffira donc de montrer que  $\text{QBF} \in \text{IP}$ , et on pourra utiliser les propriétés spécifiques de  $\text{QBF}$ , en particulier l'*arithmétisation* (on peut voir des formules booléennes comme des polynômes).

▮ **Définition 7.2.** On définit le problème  $\#\text{SAT}_D$  par

|                  |                                                                                                                                                                                                                           |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\#\text{SAT}_D$ | <p><b>Entrée.</b> Une formule booléenne <math>\phi</math> sous CNF et <math>k \in \mathbb{N}</math></p> <p><b>Sortie.</b> La formule <math>\phi</math> est-elle satisfaite par exactement <math>k</math> valuations ?</p> |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

ou, de manière équivalente,

$$\#\text{SAT}_D := \left\{ \langle \phi, k \rangle \mid \begin{array}{l} \phi \text{ est une formule booléenne satisfaite} \\ \text{par exactement } k \text{ assignations} \end{array} \right\}.$$

▮ **Théorème 7.2.** On a  $\#\text{SAT}_D \in \text{IP}$ .

▮ **Preuve.** La preuve se base sur la technique d'*arithmétisation* : on associe à  $\phi(x_1, \dots, x_n)$  un polynôme  $P_\phi(x_1, \dots, x_n)$  que l'on définit par induction :

- ▷ si  $\phi = x$ , alors  $P_\phi := x$  ;
- ▷ si  $\phi = \psi \wedge \vartheta$ , alors  $P_\phi := P_\psi \times P_\vartheta$  ;
- ▷ si  $\phi = \neg\psi$  alors  $P_\phi := 1 - P_\psi$  ;
- ▷ si  $\phi = \psi \vee \vartheta$ , alors

$$P_\phi := P_\psi + P_\vartheta - P_\psi \times P_\vartheta = 1 - (1 - P_\psi) \times (1 - P_\vartheta).$$

On définit également, pour  $i \in \llbracket 0, m \rrbracket$ ,

$$f_i(x_1, \dots, x_i) := \sum_{(a_{i+1}, \dots, a_n) \in \{0,1\}^{n-i}} P_\phi(x_1, \dots, x_i, a_{i+1}, \dots, a_n).$$

En particulier,  $f_n(x_1, \dots, x_n) = P_\phi(x_1, \dots, x_n)$ . Et, de plus,  $f_0$  est une constante qui compte le nombre d'instances qui vont satisfaire la formule  $\phi$ . On a

$$\deg f_i \leq \deg P_\phi \leq |\phi|,$$

et si  $f \neq g$  avec  $\deg f \leq d$  et  $\deg g \leq d$  alors

$$\Pr[f(x) = g(x)] \leq \frac{d}{|S|}, \quad (*)$$

si  $x$  est choisi uniformément dans  $S$ .

Maintenant, décrivons le protocole.

**Algorithme 3** Protocole interactif pour  $\sharp\text{SAT}_D$ 

| Vérificateur $\mathcal{V}$                              | Prouveur $\mathcal{P}$                                 |
|---------------------------------------------------------|--------------------------------------------------------|
|                                                         | 1 : Calculer des polynômes<br>$f_0, \dots, f_n$        |
|                                                         | $\xleftarrow{f_0}$                                     |
| 1 : <b>si</b> $f_0 \neq k$ <b>alors</b>                 |                                                        |
| 2 : $\perp$ <b>Rejeter</b>                              | $\xleftarrow{f_1(z)}$                                  |
| 3 : $y_0 := f_1(0) + f_1(1)$                            |                                                        |
| 4 : <b>si</b> $f_0 \neq y_0$ <b>alors</b>               |                                                        |
| 5 : $\perp$ <b>Rejeter</b>                              |                                                        |
| 6 : Choisir $r_1 \in S$ aléatoire-<br>ment              | $\xrightarrow{r_1}$                                    |
|                                                         | 2 : Calculer $f_2(r_1, z)$ , polynôme<br>en $z$        |
|                                                         | $\xleftarrow{f_2(r_1, z)}$                             |
| 7 : $y_1 := f_2(r_1, 0) + f_2(r_1, 1)$                  |                                                        |
| 8 : <b>si</b> $f_1(r_1) \neq y_1$ <b>alors</b>          |                                                        |
| 9 : $\perp$ <b>Rejeter</b>                              |                                                        |
| 10 : Choisir $r_2 \in S$ aléatoire-<br>ment             | $\xrightarrow{r_2}$                                    |
|                                                         | 3 : Calculer $f_3(r_1, r_2, z)$ , poly-<br>nôme en $z$ |
|                                                         | $\xleftarrow{f_3(r_1, r_2, z)}$                        |
| 11 : <i>etc</i>                                         | 4 : <i>etc</i>                                         |
| 12 : $y_n := f_n(r_1, \dots, r_n)$                      |                                                        |
| 13 : <b>Accepter</b> si $P_\phi(r_1, \dots, r_n) = y_n$ |                                                        |
| 14 : <b>Rejeter</b> sinon                               |                                                        |

Dans un premier temps, analysons la complétude du protocole. Si  $\langle \phi, k \rangle \in \sharp\text{SAT}_D$ , montrons qu'il existe  $\mathcal{P}$  qui va convaincre  $\mathcal{V}$  à coup sûr. Pour cela, il suffit de prendre  $\mathcal{P}$  qui respecte le protocole

avec les  $f_i$  définies précédemment.

Maintenant, regardons la consistance du protocole. Supposons que  $\langle \phi, k \rangle \notin \#SAT_D$ , i.e.,  $k$  n'est pas le nombre d'assignations qui vont satisfaire  $\phi$ . Montrons que tout prouveur  $\tilde{\mathcal{P}}$  a probabilité inférieure à  $1/3$  de convaincre  $\mathcal{V}$ . Lors du protocole,  $\tilde{\mathcal{P}}$  va envoyer des polynômes  $\tilde{f}_i$  qui peuvent être différents des  $f_i$ . On dira que  $\tilde{\mathcal{P}}$  est *chanceux à la phase  $i$*  si

$$f_i(r_1, \dots, r_{i-1}, z) \neq \tilde{f}_i(r_1, \dots, r_{i-1}, z)$$

mais que  $\mathcal{V}$  choisit  $r_i$  tel que

$$f_i(r_1, \dots, r_i) = \tilde{f}_i(r_1, \dots, r_i).$$

▮ **Lemme 7.1.** Si  $\langle \phi, k \rangle \notin \#SAT_D$  et si  $\tilde{\mathcal{P}}$  n'est jamais chanceux alors  $\mathcal{V}$  rejette.

▮ **Preuve.** On a que  $k = \tilde{f}_0 \neq f_0$  sinon  $\mathcal{V}$  rejette immédiatement. De même,  $\tilde{f}_0 = \tilde{f}_1(0) + \tilde{f}_1(1)$  sinon  $\mathcal{V}$  rejette à la phase suivante. On a nécessairement que  $\tilde{f}_1 \neq f_1$  car, sinon, on aurait

$$\tilde{f}_0 = \tilde{f}_1(0) + \tilde{f}_1(1) = f_1(0) + f_1(1) = f_0.$$

Puisque  $\tilde{\mathcal{P}}$  n'est pas chanceux,  $\mathcal{V}$  va choisir une valeur de  $r_1$  telle que  $\tilde{f}_1(r_1) \neq f_1(r_1)$ . On peut raisonner ainsi jusqu'à la fin du protocole. Lors de la dernière phase, on a donc

$$\tilde{f}_n(r_1, \dots, r_n) \neq f_n(r_1, \dots, r_n),$$

et, en particulier,  $\tilde{f}_n(r_1, \dots, r_n) \neq P_\phi(r_1, \dots, r_n)$ . Ainsi,  $\mathcal{V}$  va finalement rejeter (s'il n'avait pas rejeté avant).  $\square$

▮ **Lemme 7.2.** On a

$$\Pr[\exists i, \tilde{\mathcal{P}} \text{ est chanceux à l'étape } i] \leq \frac{n \times |\phi|}{|S|}.$$

▮ **Preuve.** Pour  $i \in \llbracket 1, n \rrbracket$ , on a par (\*) que

$$\Pr[\tilde{\mathcal{P}} \text{ est chanceux à l'étape } i] \leq \frac{|\phi|}{|S|},$$

et il suffit juste d'appliquer la borne de l'union. □

Désormais, il ne reste plus qu'à analyser la complexité pour vérifier que le vérificateur est bien en temps polynomial. Par la suite, on supposera que les polynômes sont à coefficients entiers. On peut vérifier que les polynômes échangés ont des coefficients de taille polynomiale. Sinon, on peut se placer sur un corps fini  $\mathbb{F}$  dans lequel les coefficients sont nécessairement de taille bornée. La seule contrainte sur  $\mathbb{F}$  est  $|\mathbb{F}| \geq |S|$ .

Le prouveur  $\mathcal{P}$  peut envoyer  $p$  tel que  $\mathbb{F} = \mathbb{Z}/p\mathbb{Z}$  à  $\mathcal{V}$  qui va vérifier que  $p$  est premier, avant toute chose, ce qui se fait en temps polynomial.<sup>1</sup> Pour tout  $S$ , on a qu'il existe un nombre premier  $p$  avec  $|S| \leq p \leq 2|S|$  qui peut être renvoyé par  $\mathcal{P}$  (*postulat de Bertrand*).

Il ne reste plus qu'à représenter  $P_\phi$ . Puisque le représenter avec des monômes peut être exponentiel (par exemple avec la formule  $\phi = \bigwedge_{i=1}^n (\neg x_i)$ ), on peut le représenter avec un circuit arithmétique ou par une formule arithmétique (*i.e.* un arbre). □

## 7.2 Preuve de $\text{IP} \subseteq \text{PSPACE}$ .

Soit  $L \in \text{IP}$  et soit  $\mathcal{V}$  le vérificateur associé.

---

1. Notamment avec le test Agrawal–Kayal–Saxena.

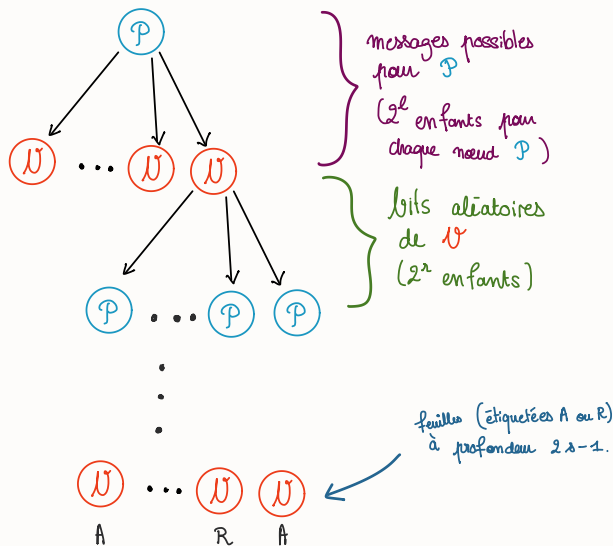
**Lemme 7.3.** On peut calculer, sur une entrée  $x$ , en espace polynomial

$$\max_{\mathcal{P}} \Pr[\mathcal{P} \text{ pousse } \mathcal{V} \text{ à accepter } x].$$

**Preuve.** On suppose que sur une entrée  $x \in \{0, 1\}^n$ ,

- ▷ le nombre de « phases » est exactement  $s := s(n)$  ;
- ▷ le vérificateur  $\mathcal{V}$  utilise exactement  $r := r(n)$  bits aléatoires à chaque phase ;
- ▷ le prouveur  $\mathcal{P}$  envoie des messages de longueurs exactement  $\ell := \ell(n)$  à chaque phase.

On peut représenter cette situation comme un arbre.



**Figure 7.2** | Arbre permettant de calculer le résultat

En chaque nœud on a une histoire partielle  $H$  qui correspond à la liste des messages envoyés par  $\mathcal{P}$  et des bits aléatoires tirés par

$\mathcal{V}$ . La *valeur* d'un nœud est

$$\max_{\mathcal{P}} \Pr[\mathcal{P} \text{ pousse } \mathcal{V} \text{ à accepter } x].$$

Pour une feuille, sa valeur est 0 ou 1.

1. Pour un nœud  $\mathcal{P}$ , on a

$$\Pr[\mathcal{V} \text{ accepte } x \mid H] = \max_{m \in \{0,1\}^n} \Pr[\mathcal{V} \text{ accepte } x \mid H \# m].$$

2. Pour un nœud  $\mathcal{V}$ , on a

$$\Pr[\mathcal{V} \text{ accepte } x \mid H] = \frac{1}{2^r} \sum_{y \in \{0,1\}^r} \Pr[\mathcal{V} \text{ accepte } x \mid H \# y].$$

On évalue la racine avec un algorithme récursif, avec une alternance de max et de moyenne.  $\square$

▮ **Corollaire 7.1.** On peut toujours supposer que  $\mathcal{P}$  fonctionne avec un espace polynomial.  $\square$

# 8 Complexité de comptage.

## 8.1 Problèmes de comptage.

▮ **Exemple 8.1.** Les problèmes  $\#SAT$ ,<sup>1</sup>  $\#CLIQUES$ ,  $\#COUPLPAR$  sont des problèmes de comptage.

|         |                                                                                                            |
|---------|------------------------------------------------------------------------------------------------------------|
| $\#SAT$ | <b>Entrée.</b> Une formule booléenne $\phi$<br><b>Sortie.</b> Le nombre de valuations qui satisfont $\phi$ |
|---------|------------------------------------------------------------------------------------------------------------|

|             |                                                                              |
|-------------|------------------------------------------------------------------------------|
| $\#CLIQUES$ | <b>Entrée.</b> Un graphe $G$<br><b>Sortie.</b> Le nombre de cliques dans $G$ |
|-------------|------------------------------------------------------------------------------|

|              |                                                                                                 |
|--------------|-------------------------------------------------------------------------------------------------|
| $\#COUPLPAR$ | <b>Entrée.</b> Un graphe biparti $G$<br><b>Sortie.</b> Le nombre de couplages parfaits dans $G$ |
|--------------|-------------------------------------------------------------------------------------------------|

▮ **Définition 8.1.** On définit  $\#P$  comme la classes des fonctions  $f : \Sigma^* \rightarrow \mathbb{N}$  telles qu'il existe  $A \in P$  et un polynôme  $p$  tel que

$$f(x) = \#\{y \in \Sigma^{\leq p(n)} \mid \langle x, y \rangle \in A\}.$$

Cette définition est analogue à **NP** pour les problèmes de comptage.

---

1. Le problème  $\#SAT_D$  vu au chapitre précédent est la version *décision* du problème  $\#SAT$  vu ici.

☞ **Définition 8.2.** Le **permanent** d'une matrice  $A$  de taille  $n \times n$  est défini comme

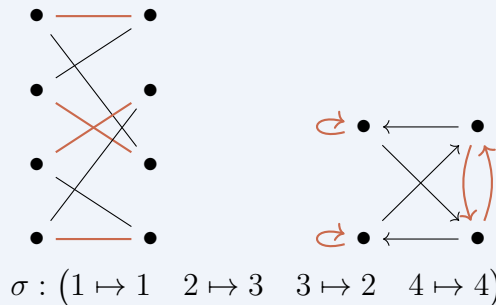
$$\text{perm}(A) := \sum_{\sigma \in \mathfrak{S}_n} a_{1,\sigma(1)} a_{2,\sigma(2)} \cdots a_{n,\sigma(n)}.$$

(Il correspond au déterminant sans le facteur  $\epsilon(\sigma)$ , signature de la permutation  $\sigma$ .)

☞ **Exemple 8.2.** Pour des matrices à entrée dans  $\mathbb{N}$ , on a  $\text{perm} \in \#P$  car

$$\text{perm}(A) = \#\{(\sigma, y) \mid \underbrace{1 \leq y \leq a_{1,\sigma(1)} a_{2,\sigma(2)} \cdots a_{n,\sigma(n)}}_{\text{se vérifie en temps polynomial}}\}.$$

On peut interpréter le permanent en combinatoire : pour une matrice  $A \in \{0, 1\}^{n^2}$ , le permanent  $\text{perm}(A)$  compte le nombre de couplages parfaits dans le graphe biparti correspondant ( $A$  est la matrice d'adjacence de  $G$ ). En effet, à une permutation  $\sigma \in \mathfrak{S}_n$  correspond à un couplage parfait comme montré ci-dessous.



**Figure 8.1** | *Interprétation combinatoire du permanent*

Le permanent compte aussi le nombre de couvertures en cycle dans les graphes orientés. Les cycles de la permutation  $\sigma \in \mathfrak{S}_n$

correspondent exactement aux cycles du graphe.

### Remarque 8.1.

- ▷ La classe  $\#P$  est « plus dure » que  $NP$  : on a  $NP \subseteq P^{\#P}$ .
- ▷ La classe  $\#P$  est « plus simple » que  $PSPACE$  : on a  $\#P \subseteq FSPACE$  où  $FSPACE$  est l'ensemble des *fonctions* calculables en temps polynomial.
- ▷ On a même que la classe  $\#P$  est plus dure que  $PH$  (théorème de Toda).

## 8.2 La $\#P$ -complétude.

**Définition 8.3.** On dit qu'une fonction  $f \in \#P$  est  **$\#P$ -complète** si, pour tout  $g \in \#P$ , il existe des fonctions  $I$  et  $O$  calculables en temps polynomial telles que

$$g(x) = O(f(I(x))).$$

Dans ce cas, je noterai  $(I, O) : g \leq_{\#} f$ .

Si  $O$  est la fonction identité alors on dit que la réduction est **parcimonieuse**.

La plupart des réductions entre problèmes (de comptages associés à des problèmes)  $NP$ -complets sont parcimonieuses. Ainsi,  $\#SAT$ ,  $\#CHEM HAM$ , *etc* sont  $\#P$ -complets par des réductions parcimonieuses.

|              |  |                                                                                           |
|--------------|--|-------------------------------------------------------------------------------------------|
| $\#CHEM HAM$ |  | <b>Entrée.</b> Un graphe $G$<br><b>Sortie.</b> Le nombre de chemins hamiltoniens dans $G$ |
|--------------|--|-------------------------------------------------------------------------------------------|

On peut composer des réductions.

▮ **Lemme 8.1.** Si  $(I_1, O_1) : f_2 \leq_{\#} f_1$  et  $(I_2, O_2) : f_3 \leq_{\#} f_2$  alors on a

$$O_2(O_1(f_1(I_1(I_2(x))))) = O_2(f_2(I_2(x))) = f_3(x),$$

d'où  $(I_1 \circ I_2, O_2 \circ O_1) : f_3 \leq_{\#} f_1$ .

▮ **Théorème 8.1.** Le problème  $\#$ CIRCUITSAT est  $\#$ P-complet.

$\#$ CIRCUITSAT | **Entrée.** Un circuit booléen  $C$  avec  $n$  variables  
**Sortie.** Le nombre de valuations qui satisfont  $C$

▮ **Preuve.** Soit  $f \in \#$ P. Il existe  $A \in \mathbf{P}$  et  $p$  tels que

$$f(x) = \#\{y \in \{0, 1\}^{p(n)} \mid \langle x, y \rangle \in A\},$$

que l'on peut réécrire en

$$f(x) = \#\{y \in \{0, 1\}^{p(n)} \mid C_n(x, y) = 1\},$$

où  $C_n$  est un circuit à  $n + p(n)$  variables, car  $A \in \mathbf{P}$ . Or, c'est une instance de  $\#$ CIRCUITSAT.  $\square$

▮ **Théorème 8.2.** Le problème  $\#$ SAT est  $\#$ P-complet.

▮ **Preuve.** On réalise une réduction parcimonieuse à partir de  $\#$ CIRCUITSAT. On introduit une variable booléenne  $z_g$  à chaque porte  $g$  du circuit. Ainsi, pour une porte  $\wedge$  sur  $g_1$  et  $g_2$ , on peut exprimer «  $z_g = z_{g_1} \wedge z_{g_2}$  » avec la formule

$$(z_g \vee \bar{z}_{g_1} \vee \bar{z}_{g_2}) \wedge (\bar{z}_g \vee z_{g_1}) \wedge (\bar{z}_g \vee z_{g_2}).$$

(C'est la même réduction que  $\text{CIRCUITSAT} \leq_{\mathbf{P}} \text{SAT}$ .) La réduction est parcimonieuse (une unique valeur possible pour chaque  $z_g$ ).  $\square$

## Est-ce que $\# \text{COUPLPAR}$ est $\# \text{P}$ -complet ?

On sait que  $\# \text{COUPLPAR}$  ne peut pas être  $\# \text{P}$ -complet pour les réductions parcimonieuses (si  $\text{P} \neq \text{NP}$ ) car sinon on aurait

$$\# \text{SAT}(\phi) = \# \text{COUPLPAR}(I(\phi)),$$

or on peut décider si  $\# \text{COUPLPAR}(I(\phi)) \geq 1$  et donc on pourrait décider  $\text{SAT}$  en temps polynomial.

▮ **Théorème 8.3 (Valiant).** Compter le nombre de couplages parfaits dans un graphe biparti est  $\# \text{P}$ -complet.

Pour montrer ce théorème, on montre d'abord que  $\text{perm}$  est  $\# \text{P}$ -complet.

Le permanent, en toute généralité, s'interprète comme

1. la somme des poids des couplages parfaits (défini comme le produit des poids des arêtes) dans un graphe biparti ;
2. la somme des poids des couvertures par cycles dans un graphe orienté.

Le *gadget du XOR* est le gadget ci-dessous.

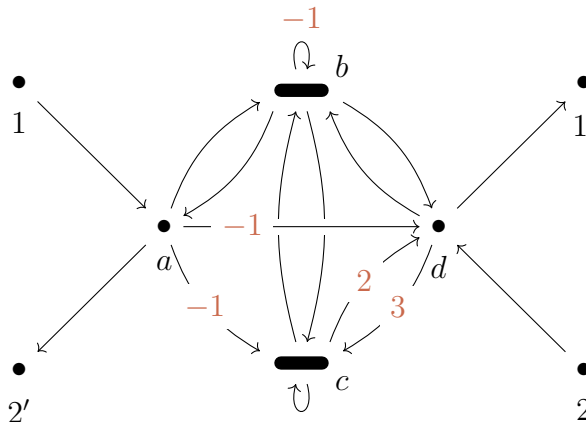
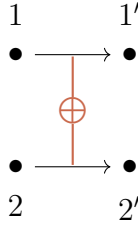


Figure 8.2 | Gadget du XOR

On s'autorisera à brancher le gadget du XOR comme ci-dessous, en retirant les arêtes  $11'$  et  $22'$ . On note  $G'$  le graphe ainsi obtenu.



On a que

$$\sum_{\substack{C \text{ couverture par} \\ \text{cycles de } G'}} \text{poids}(C) = 4 \times \sum_{\substack{C \text{ couverture par} \\ \text{cycles de } G \text{ qui} \\ \text{contient } 11' \text{ ou} \\ \text{exclusif } 22'}} \text{poids}(C).$$

On l'admet, cela pourra être vu en TD.

Le **gadget de clause** (noté  $G_c$ ) est le gadget en figure 8.3, représenté ici pour la clause  $x \vee y \vee \bar{z}$ .

La propriété du gadget de clause est la suivante : pour tout sous-ensemble strict de  $X$  de l'ensemble  $Y$  des trois arcs extérieurs, il existe une couverture par cycles unique qui contient uniquement les arcs de  $X$  parmi ceux de  $Y$ . On n'a aucune couverture par cycle qui utilise tous les arcs de  $Y$ .

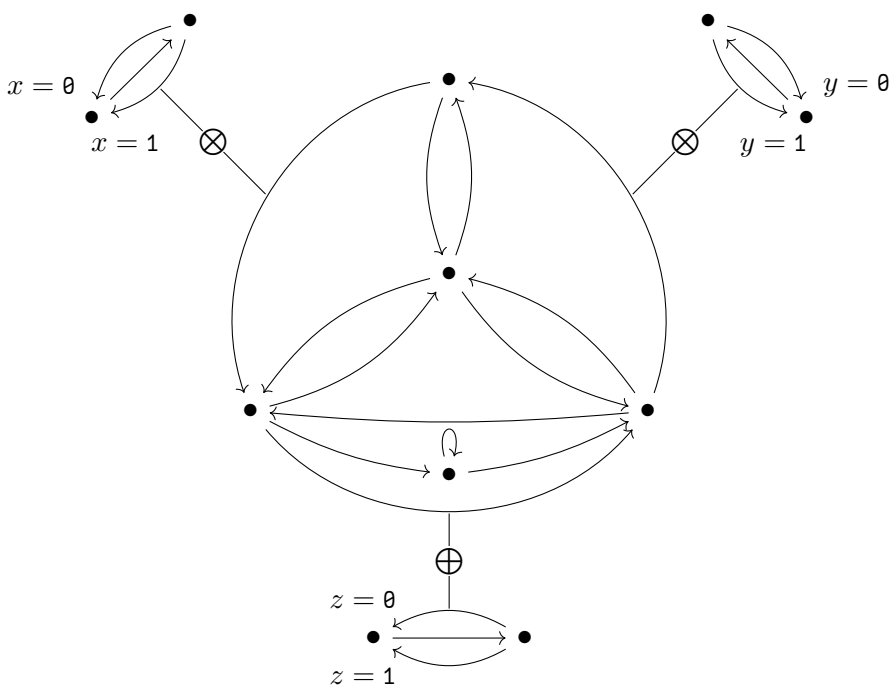
On a, pour une clause  $c$ , que

$$\sum_{\substack{C \text{ couverture par} \\ \text{cycles de } G_c}} \text{poids}(C) = 4^3 \# \text{Assignations satisfaisant } c.$$

Au final, on obtient un graphe  $G$  tel que

$$\sum_{\substack{C \text{ couverture par} \\ \text{cycles de } G}} \text{poids}(C) = 4^m \underbrace{\# \text{Assignations satisfaisant } \phi}_{\# \text{SAT}(\phi)},$$

où  $m$  est le nombre d'occurrences de littéraux dans  $\phi$ . C'est ce facteur  $4^m$  qui fait que la réduction n'est pas parcimonieuse.



**Figure 8.3** | Gadget de la clause  $x \vee y \vee \bar{z}$

☞ **Remarque 8.2.** Étant donnée une formule booléenne  $\phi$ , on construit une matrice  $A$  telle que  $\text{perm}(A) = 4^m s (*)$  où  $s = \#\text{SAT}(\phi)$ . On a  $\text{perm}(A) \bmod 3 = s \bmod 3$  ce qui est au moins aussi dur que **NP** à calculer.

Attention ! La relation  $(*)$  ne dit rien sur la complexité pour calculer le permanent modulo 2. En effet, on a

$$\text{perm}(X) \bmod 2 = \det(X) \bmod 2,$$

ce qui est calculable en temps polynomial.

### 8.3 Comptage modulaire.

☞ **Théorème 8.4 (Toda).** On a  $\text{PH} \subseteq \text{P}^{\#\text{P}}$ .

☞ **Définition 8.4.** On définit  $\oplus\text{P}$  par : un langage  $A$  est dans  $\oplus\text{P}$  ss'il existe un problème  $B \in \text{P}$  et un polynôme  $p$  tels que

$$x \in A \iff \#\{y \in \Sigma^{\leq p(n)} \mid \langle x, y \rangle \in B\} \text{ est impair.}$$

☞ **Théorème 8.5.** On a  $\text{NP} \subseteq \text{RP}^{\oplus\text{P}}.$ <sup>2</sup>

☞ **Remarque 8.3.** La même preuve montre que le comptage modulo 3 est **NP**-dur également.

☞ **Lemme 8.2 (d'isolation).** On associe à chaque élément  $x_i$  d'un ensemble  $S = \{x_1, \dots, x_n\}$  un poids entier  $w_i \geq 1$ . Soit  $\mathcal{F}$  une famille de parties de  $S$ . Pour  $T \in \mathcal{F}$ , le poids  $w(T)$  est défini par  $\sum_{i \text{ tq } x_i \in T} w_i$ . Si les  $w_i$  sont choisis indépendamment et suivant la distribution uniforme sur  $\{1, \dots, 2n\}$  alors, avec probabilité au

2. La classe **RP** a été vue en TD.

moins  $1/2$ , il existe dans  $\mathcal{F}$  un unique ensemble de poids minimaux.

▮ **Preuve.** On dit que  $x_i \in S$  est « *mauvais* » s'il appartient à un ensemble de  $\mathcal{F}$  de poids minimum mais pas à tous. Il existe un unique élément de  $\mathcal{F}$  de poids minimum ssi aucun  $x_i$  n'est mauvais.

On va montrer que, pour tout  $i$ ,

$$\Pr[x_i \text{ est mauvais}] \leq \frac{1}{2n}.$$

En effet, par borne de l'union, on peut conclure :

$$\Pr[\exists i, x_i \text{ est mauvais}] \leq \frac{n}{2n} = \frac{1}{2}.$$

On va montrer que, quels que soient les choix faits parmi les  $w_j$  avec  $j \neq i$ , si on tire  $w_i$  suivant une distribution uniforme sur  $\{1, \dots, 2n\}$  alors  $\Pr[x_i \text{ est mauvais}] \leq 1/2n$ .

Soit  $w^*(\bar{x}_i)$  le poids minimum d'un  $T \in \mathcal{F}$  qui ne contient pas  $x_i$ , et posons

$$w^*(x_i) := \min_{T \text{ tq } x_i \in T} \sum_{\substack{j \neq i \\ x_j \in T}} w_j.$$

On distingue deux cas.

- ▷ *Cas 1* :  $w^*(x_i) \geq w^*(\bar{x}_i)$ . Alors on a  $\Pr[x_i \text{ est mauvais}] = 0$  car  $x_i$  n'appartient à aucun ensemble de poids minimaux.
- ▷ *Cas 2* :  $w^*(\bar{x}_i) - w^*(x_i) \geq 1$ . On a que  $x_i$  est mauvais ssi  $w_i = w^*(\bar{x}_i) - w^*(x_i)$ 
  - Si  $w_i < w^*(\bar{x}_i) - w^*(x_i)$  alors  $x_i$  est dans tous les ensembles de poids minimaux ;
  - Si  $w_i > w^*(\bar{x}_i) - w^*(x_i)$  alors  $x_i$  n'est dans aucun ensemble de poids minimal.

Ainsi, on a que  $w_i = w^*(\tilde{x}_i) - w^*(x_i)$  est vrai avec probabilité  $1/2n$  ou 0.

□

■ **Preuve (du théorème 8.5).** Il suffit de montrer que  $\text{SAT} \in \text{RP}^{\oplus \text{P}}$ . On considère l'algorithme suivant.

**Entrée** une formule booléenne  $F(x_1, \dots, x_n)$

1 : On choisit, pour chaque  $x_i$ , un poids  $w_i \in \{1, \dots, 2n\}$ .

2 : On choisit  $u \in \{0, \dots, 2n\}$ .

3 : On note  $w(\tau) = \sum_i \text{tq } \tau(x_i)=1 w_i$  pour une assignation  $\tau$ .

4 : **si**  $\#\{\text{assignation } \tau \mid \tau(F) = 1 \text{ et } w(\tau) \leq u\}$  **est impair** **alors**

5 : | **Accepter**

6 : **sinon**

7 : | **Rejeter**

La partie rouge utilise l'oracle pour  $\oplus \text{P}$ , car on peut vérifier en temps polynomial que  $\tau(F) = 1$  et que  $w(\tau) \leq u$ .

Analysons cet algorithme.

1. Si  $F \notin \text{SAT}$  alors  $F$  est toujours refusée.
2. Si  $F \in \text{SAT}$ , on applique le lemme d'isolation à l'ensemble  $\mathcal{F}$  des  $T \subseteq \{x_1, \dots, x_n\}$  tels que l'assignation  $\tau$  définie par  $\tau(x_i) = 1$  ssi  $x_i \in T$  satisfait  $F$ . Avec probabilité au moins  $1/2$ , on a une unique assignation  $\tau$  satisfaisant  $F$  de poids minimum  $w^*$ . Si on prend  $u = w^*$ , alors

$$\#\{\tau \mid \tau(F) = 1 \text{ et } w(\tau) \leq u\} = 1,$$

cela se produit avec probabilité au moins  $1/(2n^2 + 1)$ . On a que  $F$  est acceptée avec probabilité au moins  $1/2(2n^2 + 1)$ . On peut répéter pour que la probabilité de succès soit au moins  $1/2$ .

□

Le théorème 8.5 montre que  $\text{NP} \subseteq \text{RP}^{\oplus \text{P}}$  en faisant plusieurs appels à l'oracle. Le théorème 8.6 améliore ça avec un seul appel à l'oracle.

▮ **Théorème 8.6.** On a  $\text{NP} \subseteq \text{BP}(\oplus \text{P})$ .

▮ **Définition 8.5.** Soient  $A, R \subseteq \Sigma^*$ . On dit que  $B$  est dans  $\text{BP}(A)$  s'il existe un polynôme  $p$  tel que

1. si  $x \in A$  alors,  $\#\{y \in \{0,1\}^{p(n)} \mid \langle x, y \rangle \in B\} \geq (3/4) \cdot 2^{p(n)}$  ;
2. si  $x \notin A$  alors,  $\#\{y \in \{0,1\}^{p(n)} \mid \langle x, y \rangle \notin B\} \geq (3/4) \cdot 2^{p(n)}$  ;

en notant  $n = |x|$ .

C'est une réduction probabiliste de  $B$  à  $A$ .

Soit  $\mathcal{C}$  une classe de complexité. On dit que  $B \in \text{BP}(\mathcal{C})$  s'il existe  $A \in \mathcal{C}$  tel que  $B \in \text{BP}(A)$ , autrement dit,

$$\text{BP}(\mathcal{C}) := \bigcup_{A \in \mathcal{C}} \text{BP}(A).$$

▮ **Remarque 8.4.** Les principales étapes de la preuve du théorème de Toda sont les deux inclusions

$$\Sigma_k^{\text{P}} \subseteq \text{BP}(\oplus \text{P}) \subseteq \text{P}^{\# \text{P}}.$$

La deuxième inclusion demande de *supprimer l'aléatoire*.

▮ **Proposition 8.1.** On a  $\Sigma_k^{\text{P}} \subseteq \text{BP}(\oplus \text{P})$ .

▮ **Preuve (idée de la preuve).** Par récurrence sur  $k$ , on montre que  $\Sigma_k^{\text{P}} \subseteq \text{BP}(\oplus \Pi_{k-1}^{\text{P}})$  (pour le cas  $k = 2$ , c'est le théorème 8.6). On a que  $\Pi_{k-1}^{\text{P}} \subseteq \text{BP}(\oplus \text{P})$ , et  $\text{BP}(\oplus \text{P})$  est clos par complément. En effet, on a donc

$$\Sigma_k^{\text{P}} \subseteq \text{BP}(\oplus \Pi_{k-1}^{\text{P}}) \subseteq \text{BP}(\oplus \text{BP}(\oplus \text{P})) = \text{BP}(\oplus \text{P}).$$

□

▮ **Lemme 8.3.** On a que  $A \in \oplus \mathbf{P}$  ss'il existe un polynôme  $p$  et un problème  $B \in \mathbf{P}$  tel que, pour tout  $x \in \Sigma^*$ ,

$$x \in A \iff \#\{y \in \{0,1\}^{p(|x|)} \mid \langle x, y \rangle \in B\} \text{ est pair.}$$

Ainsi,  $\oplus \mathbf{P}$  est clos par complément.

▮ **Preuve.** Il existe  $C \in \mathbf{P}$  et un polynôme  $q$  tels que, pour toute entrée  $x \in \Sigma^*$ ,

$$x \in A \iff \#\{y \in \{0,1\}^{q(|x|)} \mid \langle x, y \rangle \in C\} \text{ est impair.}$$

On définit  $B$  par :

$$B := \left\{ \langle x, y \rangle \mid \begin{array}{l} y \neq 0^{q(|x|)} \wedge \langle x, y \rangle \in C \\ \text{ou} \\ y = 0^{q(|x|)} \wedge \langle x, y \rangle \notin C \end{array} \right\}.$$

« On inverse la réponse sur  $y = 0^{q(|x|)}$ . »

□

▮ **Lemme 8.4.** Soit  $A \in \oplus \mathbf{P}$ . L'ensemble

$$A' := \left\{ \langle x_1, \dots, x_k \rangle \mid \bigvee_{i=1}^k (x_i \in A) \right\}$$

est dans  $\oplus \mathbf{P}$ .

▮ **Preuve.** Par le lemme précédent, il existe  $p$  un polynôme et  $B \in \mathbf{P}$  tel que,

$$x \in A \iff \#\{y \in \{0,1\}^{p(|x|)} \mid \langle x, y \rangle \in B\} \text{ est pair.}$$

On pose  $B'$  l'ensemble

$$B' := \left\{ \langle x_1, \dots, x_k, y_1, \dots, y_k \rangle \mid \bigwedge_{i=1}^k (\langle x_i, y_i \rangle \in B) \right\}.$$

On a

$$\begin{aligned}
 & \langle x_1, \dots, x_k \rangle \in A' \\
 \iff & \bigvee_{i=1}^k \# \{y_i \in \{0, 1\}^{p(|x|)} \mid \langle x_i, y_i \rangle \in B\} \text{ pair} \\
 \iff & \# \{ \langle y_1, \dots, y_n \rangle \mid \langle x_1, \dots, x_k, y_1, \dots, y_k \rangle \in B \} \text{ pair} \\
 & .
 \end{aligned}$$

□

▮ **Preuve (du théorème 8.6).** On montre que  $\text{SAT} \in \text{BP}(\oplus \mathbf{P})$ . Soit  $F(x_1, \dots, x_n)$  une formule booléenne et soit  $m = 2(2n^2 + 1)$ . On accepte  $F$  ssi

$$\langle F, w^{(1)}, u_1, w^{(2)}, u_2, \dots, w^{(2m)}, u_{2m} \rangle$$

(où les  $w^{(i)}$  et les  $u_i$  sont tirés au sort) satisfait la propriété

$$\bigvee_{j=1}^{2m} \# \{ \tau \in \{0, 1\}^n \mid \tau(F) = 1 \text{ et } w^{(j)}(\tau) \leq u_j \} \text{ est impair,}$$

et ce dernier est dans  $\oplus \mathbf{P}$  par le lemme précédent.

Analysons cet algorithme.

- ▷ Si  $F \notin \text{SAT}$ , alors on accepte  $F$  avec probabilité 0 car les  $2m$  expériences échouent.
- ▷ Si  $F \in \text{SAT}$ , alors on a

$$\Pr[F \text{ rejetée}] \leq \left(1 - \frac{1}{m}\right)^{2m} \leq \frac{1}{e^2},$$

d'où  $\Pr[F \text{ acceptée}] \geq 3/4$ .

□